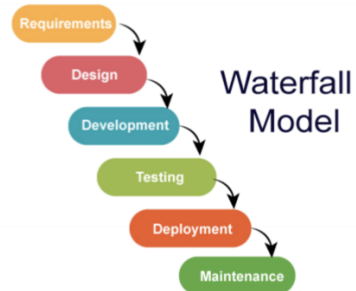


The Old Model: Waterfall

Optimized for predictability, not change.

- Sequential phases (Requirements → Design → Code).
- Assumes **100% stable requirements** upfront.
- The Flaw: **Testing happens last**. Fixing a bug at the end is **100x more expensive** than fixing it in design.



[Waterfall model \(Hello PM\)](#)



Agile vs. Scrum

Agile (Philosophy)

- A set of **values and principles**.
- Focuses on adaptability and speed.
- Not a specific set of rules.

Scrum (Framework)

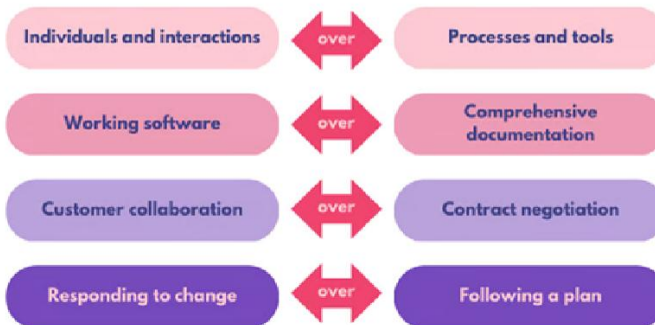
- A **concrete implementation** of Agile.
- Defines Roles, Events, and Artifacts.
- Gives teams a 'Process Operating System'.

[Agile vs Scrum \(Atlassian\)](#)

The Agile Manifesto

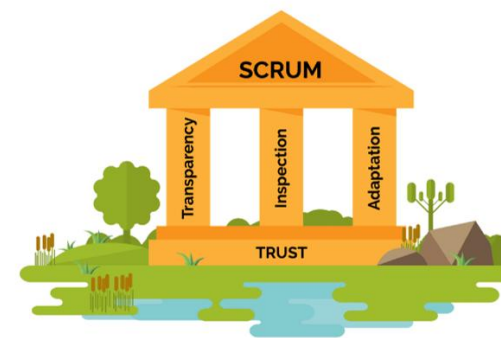
We value items on the left **MORE** than items on the right

4 Agile Manifesto values



[The Agile Manifesto](#)

Scrum Values



- COURAGE**
Scrum Team members have courage to do the right thing and work on tough problems
- FOCUS**
Everyone focuses on the work of the Sprint and the goals of the Scrum Team
- COMMITMENT**
People personally commit to achieving the goals of the Scrum Team
- RESPECT**
Scrum Team members respect each other to be capable, independent people
- OPENNESS**
The Scrum Team and its stakeholders agree to be open about all the work and the challenges with performing the work

Credit: ABN AMRO Bank N.V.

[Scrum Values \(scrum.org\)](#)

Scrum Framework



[Breaking down the Agile framework \(Atlassian\)](#)

88 Software Development Terms to Know

Learn the essential terms to know for a software development project.

A

- **Acceptance Testing:** The testing that occurs on the client side.
- **Adaptive maintenance:** If a program changes to a different operating system or environment, this occurs.
- **Agile Software Development:** A development approach that involves flexible planning and continuous improvement to deliver the software as fast as possible.
- **Algorithm:** A sequence of steps that will accomplish a task if followed correctly.
- **Alpha testing:** The testing that occurs within a software development company.
- **API (Application Programming Interface):** A tool that provides completed building blocks with functionality that can be put together and into a software.

B

- **Backend:** A 'server' software that users do not interact with directly.
- **Beta testing:** The testing that happens using client or selected personas' feedback.
- **Bottom-up design:** An area of program refinement that starts with smaller modules and builds into a complete software program.
- **Bugs:** A program error.

C

- **Code Refactoring:** The process of editing code to reduce or eliminate technical debt.
- **Compiler:** A program that translates a high-level program into an independent one.
- **CI (Continuous Integration):** The practice of building software when all working copies of code are merged into one branch.
- **Corrective Maintenance:** Takes place after a software is installed if previously undetected errors pop up.
- **CSS (Cascading Style Sheets):** A style sheet language that organizes HTML elements.

D

- **Data Abstraction:** A principle of data modeling that emphasizes the separation between the external interface and internal implementation.
- **Data Modeling:** A process used in object-oriented languages that identify objects and how they relate.
- **Data Structures:** A specialized format for organizing and processing data.
- **Debugging:** The detection and removal of errors in a program.
- **Dependencies:** Correlations between tasks that show in which order they should be executed.
- **Deploy:** Inputting the code into the server so it can run.

E

- **End User:** The individual that uses a particular product.

- **Event Driven:** A system that responds to an external event.
- **Event Driven Language:** Designed to handle external events.
- **Executable Code:** Code that can be run independently without translation.

F

- **Frontend:** A 'client' software that users interact with.
- **Functional Language:** A language that uses the evaluations of expressions rather than commands.
- **Full Stack:** A software developer that can work on the frontend and backend.
- **Framework:** A set of templates for a programming platform.

G

- **General Purpose Language:** A language that can be used to program solutions that cover a range of situations.
- **Git:** A system for programmers to manage code content and other versions.

H

- **Hardware:** Physical parts of a computer or another electronic device.
- **High-level Language:** A language that is easily understood by programmers and development teams.
- **HTML (Hypertext Markup Language):** A markup language for the web that structures web pages.
- **Human Computer Interface:** The interface that allows the program to interact with the outside world.

I

- **IDE (Integrated Development Environment):** An [application](#) that facilitates software developers with writing code.
- **Independent Test Group:** Testing of software by a group outside of the development team.
- **Intermediate Code:** Compiled code that is produced for a target computer.

- **ISO:** A quality assurance system for software development.
- **Iterative:** An iterative process that involves feedback and repetition.

Additional Reading: '[How to Budget for Custom Software Development \[With Template\]](#)'

J

- **Javascript:** A portable language for a wide range of platforms.

K

- **Kanban:** An approach for managing software development that depicts all planned work to optimize the overall process.

L

- **Linear Search:** A standard algorithm that performs searches on data items in sequential order.

M

- **Machine Code:** Native computer code that can be understood without translation.
- **Machine Learning:** The study of algorithms and models that learn from data sets to make predictions or decisions on new data.
- **Maintenance:** The upkeep of a software program.
- **Markup Language:** A text-encoding system that consists of symbols and notations to control a structure or relationship within a program.
- **Methodology:** A technique that enables the design and development of software to be implemented.
- **Metric:** In software, the measure of whether a product feature or task is quantifiable or countable.

N

- **Normal Operation:** Running of a program under expected normal conditions.

O

- **Object:** A data item that can be manipulated by a computer program or system.

- **Object-oriented Programming:** A programming system based on the idea that objects that contain both data and code can modify data.
- **Open Source:** Source code that's freely available for making changes while being free to download.
- **Operating System:** The software system that supports a computer or product's basic functions.

P

- **Performance Testing:** A form of testing that examines the speed, response time, and usage of software under a particular workload.
- **Programmers:** A section of a development team that's responsible for coding, testing, implementation, and maintenance.
- **Programming Language:** A system of notations for writing software programs.
- **Project Manager:** Member of the development team that supervises the project.
- **Prototype:** A simple working version of a software.
- **Plugin:** An add-on that adds new functionalities to a host program or software without alternating the host itself.
- **Pull Request:** A notification given by developers when they're done building a feature.

Q

- **QA (Quality Assurance):** The process of reviewing a software product to ensure it meets expectations.

R

- **Relational Database:** A database structured to recognize relations among any stored items or objects.
- **Regression Testing:** The process of re-running tests to verifying the correctness of software after a change.
- **Repository:** A place where data is gathered and protected.
- **Runtime:** The length of time a program takes to run.

S

- **SaaS (Software as a service):** Type of software delivery model that's provided over the Internet to the end user by the software owner on demand.
- **Scripting Language:** Used for writing smaller programs or scripts that enhance existing software.
- **Scrum:** A project management methodology for effective team collaboration that's based on Agile software development.
- **SDLC (Software Development Life Cycle):** The general process of software development that follows a series of steps until completion.
- **Software:** Programs that help a computer system execute tasks.
- **Software Engineering:** An area of computing where the emphasis is on the development of high-quality software.
- **Source Code:** The code for a program that's written in a high-level language that is then translated into machine code.
- **Syntax:** The structure or grammar of a statement in a given language.

T

- **Testing:** Running the program to ensure its reliability.
- **Test Plan:** A strategy that involves testing software under different conditions.
- **Top-Down Design:** A design approach that involves putting larger systems into smaller components.

U

- **Unit Testing:** Testing specific pieces of code in a system.
- **Usability:** A quality system to determine how easy something is to use.
- **User experience:** Software design that focuses on usability and user satisfaction.
- **User interface:** Software design that focuses on the looks and style.

V

- **Version Control:** A software system used to manage changes to a codebase over time to help recall specific versions later.

W

- **Waterfall Model:** One of the earliest models for software development that incorporates seven stages.
- **Web App:** An application program on a server that is delivered over the Internet.
- **Website:** Collection of web pages that share a domain name.

X

- **XML (Extensive Markup Language):** A markup language for encoding documents and transmitting them into serialized data.

Y

- **YAML:** A human-readable data format for storing and transmitting information.

Z

- **Z Object Publishing Environment:** An open-source set of tools for publishing components within different platforms.

2

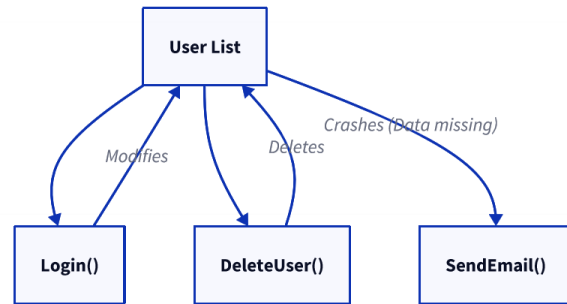
The "Systems" Problem

Why scripts don't scale

If changing one feature forces you to edit 10 unrelated files, your **design** is the problem, not your coding skills.

The Problem

- Scripts work for 100 lines. They fail at 10,000 lines
- Data is often global or passed endlessly between functions.
- It becomes hard to track who is changing what.
- Distributed systems have multiple modules talking over networks



4/26

The Solution: Object Oriented Design

OOP changes the mental model. Instead of "Functions" and "Variables", we have **Objects**.

It helps you manage:

- **Complexity** (hide details behind clean boundaries)
- **Change** (localize edits; avoid ripple effects)
- **Integration risk** (clear contracts between modules/teams)
- **Reuse without copy-paste** (stable interfaces + swappable implementations)

5/26

Classes vs Instances (Python)

A class is a blueprint

- A logical grouping of data (**attributes**) and functions (**methods**).
- It defines a new data type but occupies no "real" space in your program's logic until it is used.

Instance (The Object)

- A concrete manifestation of a class.
- It is a live "thing" in your computer's memory that has its own unique data.

```
class Dog:
    species = "Canis familiaris" # Class attribute (Shared)

    def __init__(self, name, breed): # The constructor. Sets up the initial state.
        self.name = name            # Instance attribute (Unique)
        self.breed = breed          # Instance attribute (Unique)

buddy = Dog("Buddy", "Golden Retriever") # Instance #1 (The Objects)
miles = Dog("Miles", "Greyhound") # Instance #2 (The Objects)
```

- **Q:** What would the values for buddy.breed and buddy.species be?
- **Q:** What does self keyword do?

7/26

The 4 Pillars of OOP

These are the core concepts we will cover today.

1. Encapsulation:

- Hides internal details; provides controlled access via methods or properties

2. Polymorphism:

- Different classes can define the same method name but implement it differently.

3. Abstraction:

- Expose only essential details; hide complex implementation.

4. Inheritance:

- Reuse existing code by creating subclasses from parent classes.

10/26

Inheritance vs. Composition

How to build big things

Inheritance ("Is-A")

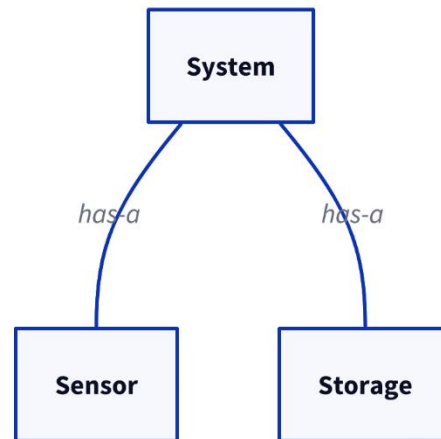
- Car is a Vehicle

Composition ("Has-A")

- Car has an Engine

Preferred:

- **Hot-Swapping:** Change behavior at runtime. You can swap the Engine easily.
- **Testability:** Pass in mock components during tests.
- **Simplicity:** No complex method resolution order (MRO).



17/26

Inheritance: The Coupling Trap

Use "Is-A" sparingly; it creates rigid bonds

The Fragile Base Class

Inheritance creates the tightest coupling available in OO design.

- **The Problem:** If you change the Parent class, you inadvertently break all Child classes.
- **The "God Object":** Base classes tend to grow huge as they accumulate shared logic for everyone.
- **Rule of Thumb:** Prefer Composition unless the relationship is a permanent, logical "Is-A" (e.g., Dog is an Animal).

```
# The Components
class LiIonBattery:
    def power(self): return "12V"

class NuclearBattery:
    def power(self): return "5000V"

# The System (Composed)
class Robot:
    def __init__(self, battery):
        # We inject the dependency
        self.battery = battery

    def start(self):
        p = self.battery.power()
        print(f"Powered by {p}")

# Swapping components is easy!
r1 = Robot(LiIonBattery())
r2 = Robot(NuclearBattery())
```

18/26

Error Handling: Don't Crash

Using `try`, `except`, and `finally` to stay alive

The "Safe" Pattern

Instead of letting an error kill your program, we "catch" it.

1. `try`: Attempt the dangerous code.
2. `except`: Define what to do if it fails (e.g., use a default value).
3. `finally`: Cleanup code that **always** runs (like closing files), even if an error occurred.

```
def load_settings(filename):  
    file = None  
    try: # Dangerous: File might not exist  
        print("Opening file...")  
        file = open(filename, 'r')  
        data = file.read()  
        return data  
  
    except FileNotFoundError: # Handle specific error  
        print("Error: File missing. Using defaults.")  
        return "Default Config"  
  
    except Exception as e: # Catch generic unexpected errors  
        print(f"Something weird happened: {e}")  
  
    finally: # Cleanup (Runs even if we returned above!)  
        print("Cleanup: Closing file handler.")  
        if file:  
            file.close()
```

What are Requirements?

The Definition:

Requirements are the descriptions of the **services** a system should provide and the **constraints** on its operation.

They reflect the needs of customers for a specific purpose, such as:

- Controlling a device (e.g., IoT Controller).
- Placing an order (e.g., eCommerce).
- Finding information (e.g., Search Engine).



What is Requirement Engineering (RE)?

It is the systematic process of:

1. **Finding out** (Elicitation)
2. **Analyzing** (Negotiation)
3. **Documenting** (Specification)
4. **Checking** (Validation)

RE is how we turn a vague "need" into a buildable "spec".

Distinguishing the Levels

User vs. System

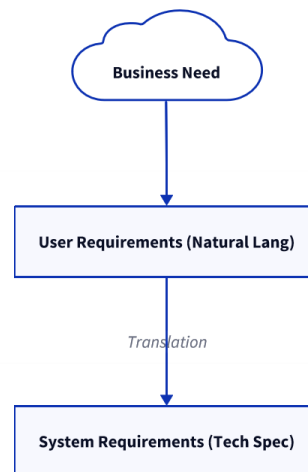
To solve the ambiguity, we classify requirements into two distinct levels:

1. User Requirements:

- **Format:** Natural language + diagrams.
- **Audience:** Customers, Managers, Users.
- *Broad statements of features.*

2. System Requirements:

- **Format:** Detailed, structured text.
- **Audience:** Developers, Testers.
- *The "Contract" for the build.*



🧠 Activity: Prioritization (MoSCoW)

We cannot build everything. We must prioritize features. This is the main tool for **Analysis**.

Scenario: Developing a **Distributed Chat Application**.

Priority	Definition	Example
Must Have	Critical. If missing, project fails.	Send/Receive Messages.
Should Have	Important but can wait.	Message Timestamps.
Could Have	Desirable. "Nice to have".	Dark Mode / Emojis.
Won't Have	Agreed to drop for now.	Voice Calling.

Functional vs. Non-Functional

The "What" vs. The "How"

Functional Requirements (FR):

- Interactions between system and environment.
- **Deterministic:** Given Input X, System does Y.
- *Examples:* Calculations, Data processing, API endpoints.

// methods & functions
• FR-failure results in system failure

Non-Functional Requirements (NFR):

- Constraints on the services or functions.
- Also known as "Quality Attributes" (The "ilities").
- *Examples:* Reliability, Scalability, Security, Maintainability.

// The whole architecture... I believe
• NFR-failure results in an unhappy customer or worse

Question 2 (2 points)

You are leading a team to build a distributed food delivery platform. The initial stakeholder interviews reveal the following raw inputs:

- "Customers need to order food easily."
- "Restaurants should see orders in real-time."
- "Drivers need to know where to go."
- "The system must be fast and reliable."
- "We want AI recommendations eventually."

Classify each of the five statements as either a User Requirement, a System Requirement, a Functional Requirement, or a Non-Functional Requirement. Some statements may require transformation before classification.

from https://learn.bck.ca/2020/05/05/using-user/attempt/question_start_frame_auth_auth-11898188.aspx?gq=1074900&fpl=05.dnb-08.frame08-08.unhappiness-08

- "Customers need to order food easily."
 - Transform: Customers need an UI that classifies each section of the menu and allows them to choose exactly what they want, how much, and pay for the cart at the end.
 - Class: User requirement
- "Restaurants should see orders in real-time."
 - Transform: Restaurant's management need an app that can see confirmed order within a few seconds of their confirmation/payment at the customer's end.
 - Class: Functional requirement
- "Drivers need to know where to go."
 - Class: functional requirement
- "The system must be fast and reliable."
 - Class: Non-functional requirement (reliability, speed, efficiency)
- "We want AI recommendations eventually."
 - System Class: requirements

Transition to Agile

From Documents to Backlogs

In **Waterfall**, we write a huge specification document.

In **Agile/Scrum**, we acknowledge that requirements change. We use a **Product Backlog**.

The Hierarchy:

1. **Epics:** Large chunks of work (Modules).
2. **User Stories:** Small, valuable features.
3. **Tasks:** Technical steps (Implementation details).

Epics vs. Features

Grouping by Service

Epics (The Modules):

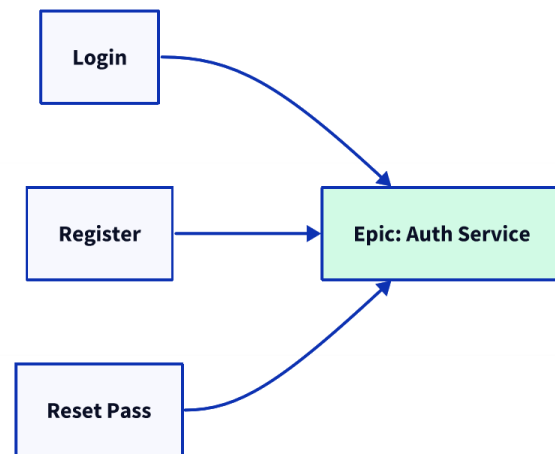
Large bodies of work. In Distributed Systems, these map to your major Software Modules.

- *Example:* "Order Management System" or "Authentication Service".

Features/Stories (The Logic):

Functional components that fit in a sprint.

- *Example:* "As a user, I want to login."



User Stories & INVEST

The Unit of Delivery

Format:

As a <ROLE>, I want <FEATURE>, so that <BENEFIT>.

The "Benefit" prevents building useless features.

Quality Check (INVEST):

- Independent
- Negotiable
- Valuable
- Estimable (Can we guess the time?)
- Small (Fits in a sprint?)
- Testable

Acceptance Criteria (AC)

The "Definition of Done".

Story: Upload Profile Picture.

AC:

1. Supports JPG/PNG.
2. Max size 5MB.
3. Error if upload fails.

A story is not done until ACs pass.

Summary: The Engineering Mindset

1. Ambiguity is the Enemy:

RE removes "I guess" from the development process.

2. Process Matters:

Elicit (Ask) → Analyze (Negotiate/MoSCoW) → Validate (Check) → Manage (Trace).

3. Agile Structure:

Use **Epics** for system modules, **Stories** for features, and **ACs** for verification.

4. Next Step:

Go to the Lab. Form Teams. Ideate. Create your Backlog.



3

From Requirements to Architecture

Last Week (Requirements):

We focused on the **What** and **Why**.

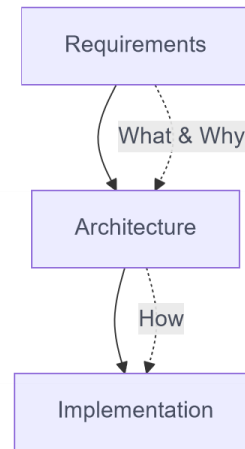
- "The system shall allow users to send messages."
- "The system shall support 100 concurrent users."
- "The user wants to share files with teammates."

This Week (Architecture):

We focus on the **How**.

- "We will use a Client-Server pattern."
- "Messages will be stored in a PostgreSQL database."
- "The server will expose a REST API."

The Transition:



Requirements tell us what to build.

Architecture tells us how to organize it.

Why Architecture Matters

The Cost of Change:

Decisions made during architecture are **expensive to reverse** later.

WHEN YOU FIND A PROBLEM	RELATIVE COST TO FIX
During Requirements	1x
During Architecture	5x
During Implementation	10x
During Testing	25x
After Deployment	100x

Spending time on architecture saves time on debugging.

Good Architecture Enables:

- **Scalability:** Can handle growth
- **Maintainability:** Easy to change later
- **Testability:** Components can be tested independently
- **Security:** Clear boundaries for access control
- **Communication:** Team understands the structure

Example of a Bad Decision:

You build a chat app where the client connects directly to the database. Later, you realize:

- No security layer exists
- 1000 users crash the database
- You must rewrite everything

Four Perspectives on System Modeling

To fully understand a system, we look at it from **four different angles**. Each perspective answers a different question and uses a different diagram type.

PERSPECTIVE	QUESTION ANSWERED	DIAGRAM TYPE	FOCUS
Context	Where does the system end?	Context Diagram	Boundaries and external actors
Interaction	How do parts communicate?	Sequence Diagram	Message flow over time
Structure	What are the parts?	Component Diagram	Organization and dependencies
Behavior	How does it react to events?	State Diagram	State changes over time

Key Insight:

No single diagram tells the whole story. You need multiple perspectives to fully document a system.

We will now explore each perspective in detail.

Perspective 1: Context Models

Q: Where does your system end and the outside world begin?

A **context model** is a high-level view of your system and its relationships with external entities.

Key Concepts:

TERM	DEFINITION	EXAMPLE
System	What you are building	Your Chat App
Actor	External entity that interacts	User, Admin
External System	Service you don't control	Gmail API, Stripe

Why This Matters:

If you don't define the boundary:

- You might try to build things that already exist
- You might forget to integrate with critical services
- Scope creep becomes uncontrollable

Context Diagram Example

The context diagram is your first line of defense against scope creep.

Scenario: Distributed Chat Application

Inside the System:

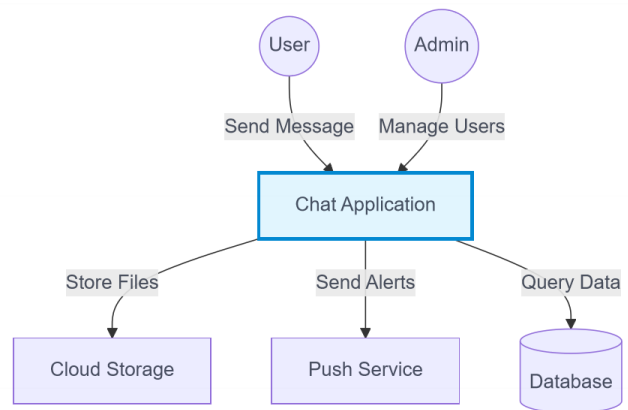
- Message handling logic
- User authentication
- File storage management

Outside the System:

- Users (interact via browser/app)
- Push notification service
- Cloud file storage

Reading This Diagram:

- The blue box is your system
- Circles are human actors
- Rectangles are external services
- Arrows show data/control flow



Perspective 2: Interaction Models

Q: How do components communicate to complete a task?

An **interaction model** shows how components exchange messages to accomplish a task, capturing the order and flow of communication.

Key Concepts:

TERM	DEFINITION
Participant	A component or actor in the interaction
Message	Data or command sent between participants
Lifeline	Vertical line showing a participant's existence over time
Response	Return message (often dashed arrow)

When to Use Sequence Diagrams:

- Documenting API call flows
- Explaining authentication processes
- Showing how a user action triggers system behavior
- Debugging complex multi-component interactions

Sequence diagrams are essential for distributed systems because they show network communication.

Sequence Diagram Example

Scenario: User Login Flow

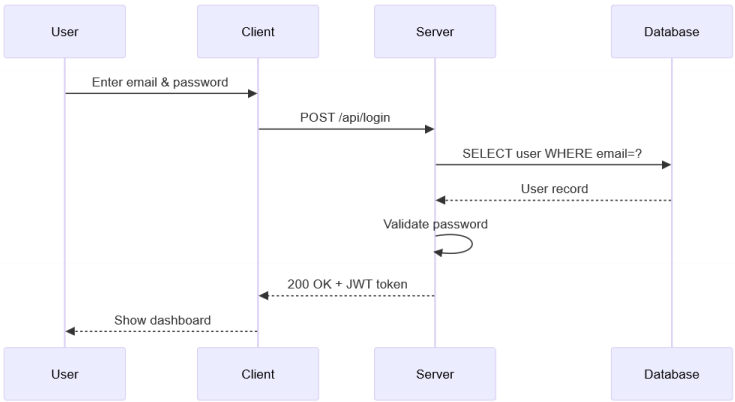
1. User enters credentials in the client

2. Client sends login request to server

3. Server queries the database
4. Database returns user data

5. Server validates and returns a token

6. Client stores token and shows success



Perspective 3: Structural Models

Q: What are the major parts of the system and how are they organized?

A **structural model** shows the static organization of your system—the components, their responsibilities, and how they depend on each other.

Key Concepts:

TERM	DEFINITION
Component	A modular unit with a defined interface
Dependency	One component relies on another
Layer	A horizontal grouping of related components
Interface	The contract for how components communicate

Structural vs. Interaction:

STRUCTURAL MODEL	INTERACTION MODEL
Shows organization	Shows communication
Static view	Dynamic view
"What exists"	"What happens"
Component diagram	Sequence diagram

Think of structural models as the "org chart" for your code.

Component Diagram Example

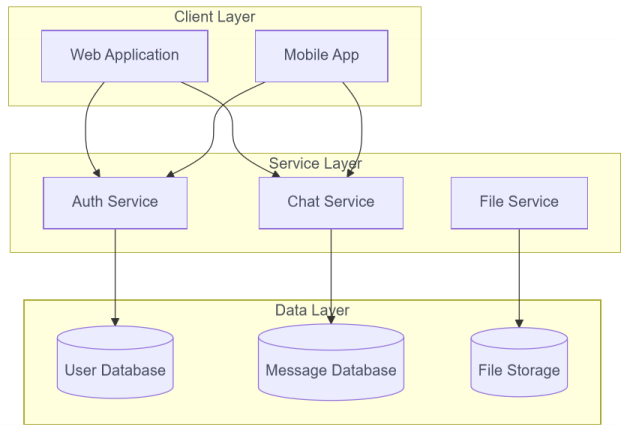
Scenario: Chat Application Structure

Components Identified:

- **Client Layer:** Web UI, Mobile App
- **Service Layer:** Auth, Chat, File services
- **Data Layer:** User DB, Message DB, File Storage

Reading This Diagram:

- **Subgraphs** group related components
- **Arrows** show dependencies (A depends on B)
- **Cylinders** represent databases



Perspective 4: Behavioral Models

Q: How does the system react to events and change state?

A **behavioral model** captures how a system responds to events by changing states—documenting the conditions, triggers, and transitions that define its dynamic behavior.

Key Concepts:

TERM	DEFINITION
State	A condition the system is in
Transition	Movement from one state to another
Event	Something that triggers a transition
Guard	A condition that must be true for transition

When to Use State Diagrams:

- User session management (logged in/out)
- Order processing (pending, paid, shipped)
- Connection states (connecting, connected, disconnected)
- Game states (menu, playing, paused, game over)

State diagrams help you think about edge cases and error handling.

State Diagram Example

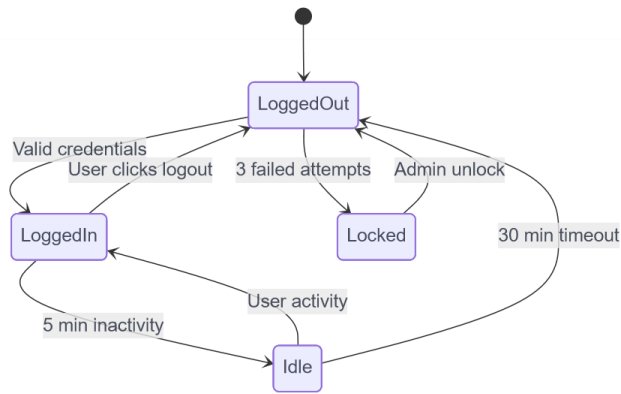
Scenario: User Session States

States Identified:

- Logged Out (initial state)
- Logged In (active session)
- Idle (inactive but not logged out)
- Locked (too many failed attempts)

Reading This Diagram:

- **[*]** is the starting point
- **Rectangles** are states
- **Arrows** are transitions with triggering events
- Follow the arrows to trace possible paths



Summary: Four Perspectives

Each perspective provides a unique view of your system. Together, they form a complete picture.

PERSPECTIVE	DIAGRAM	QUESTION	USE WHEN
Context	Context Diagram	What is inside vs. outside?	Defining scope
Interaction	Sequence Diagram	How do parts communicate?	Documenting API flows
Structure	Component Diagram	What are the parts?	Organizing code
Behavior	State Diagram	How does it react?	Handling complex workflows

For Your Project (Lab 4):

- **Required:** Context Diagram, Component Diagram
- **Recommended:** At least one Sequence Diagram for a core flow
- **Optional:** State Diagram if your system has complex states

Now let's look at how to organize these components using Architectural Patterns.

Architectural Patterns

An architectural pattern is a **reusable solution** to a commonly occurring problem in software architecture.

Why Use Patterns?

BENEFIT	EXPLANATION
Proven	Used successfully in many systems
Vocabulary	Common language with other developers
Guidance	Provides a starting structure
Trade-offs	Known strengths and weaknesses

Patterns We Will Cover:

- 1. **Client-Server** — The foundation of web applications
- 2. **Layered (N-Tier)** — Separation of concerns
- 3. **Microservices** — Independent deployable services

Choosing a pattern is one of the most important architectural decisions.

Pattern 1: Client-Server

Structure:

The system is divided into two roles:

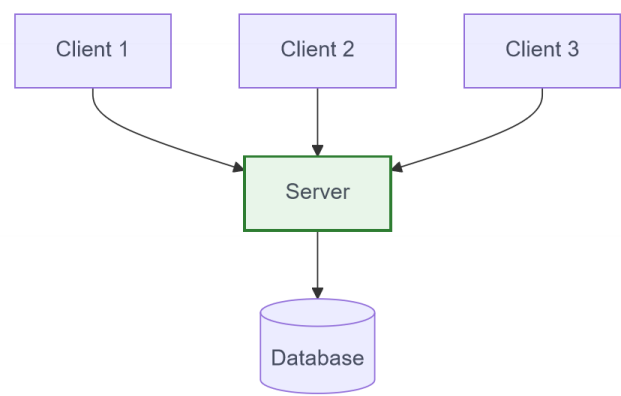
- **Client:** Requests services (browser, mobile app)
- **Server:** Provides services (web server, API)

Characteristics:

- Clients initiate communication
- Server is the central point of control, contains data
- Multiple clients connect to one server

Trade-offs:

PROS	CONS
Simple to understand	Server is single point of failure
Centralized data	Server can become bottleneck
Easy to secure	Requires network connection



Examples:

- Web applications (React + Flask)
- Mobile apps connecting to REST APIs
- Email clients connecting to mail servers

Pattern 2: Layered (N-Tier)

Structure:

System is organized into layers, each with a specific responsibility.

Common Layers:

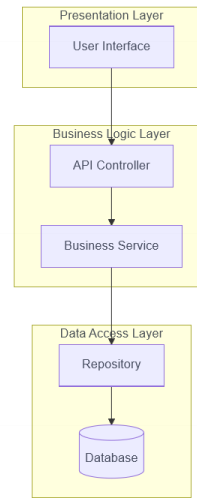
1. **Presentation:** User interface (what users see)
2. **Business Logic:** Rules, calculations, workflows
3. **Data Access:** Database queries and storage

Rules:

- Each layer only talks to adjacent layers
- Layers can be replaced independently

Trade-offs:

PROS	CONS
Clear separation	More initial setup
Easier testing	Potential overhead
Maintainable	Can become rigid



Examples:

MVC frameworks (Flask), Enterprise apps, E-commerce systems

Pattern 3: Microservices

Structure:

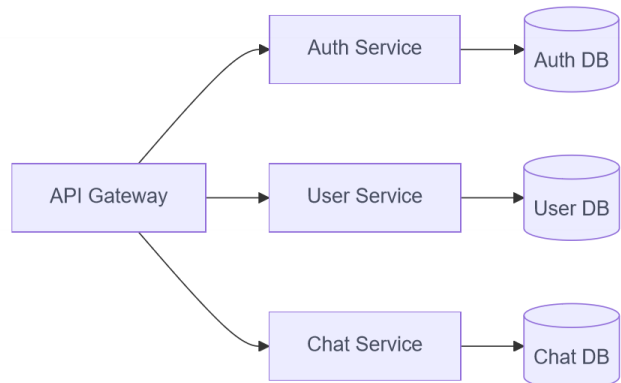
The system is composed of many small, independent services that communicate over a network.

Characteristics:

- Each service has a single responsibility
- Services are independently deployable
- Each service can have its own database
- Services communicate via APIs

When to Use:

- Large teams working on different features
- High scalability requirements
- Need to use different technologies per service



Comparing Patterns

Which Pattern Should You Choose?

FACTOR	CLIENT-SERVER	LAYERED	MICROSERVICES
Complexity	Low	Medium	High
Scalability	Moderate	Good	Excellent
Development Speed	Fast	Medium	Slow initially
Team Size	Small (1-3)	Medium (3-6)	Large (6+)
Best For	Simple web apps	Business apps	Large-scale systems

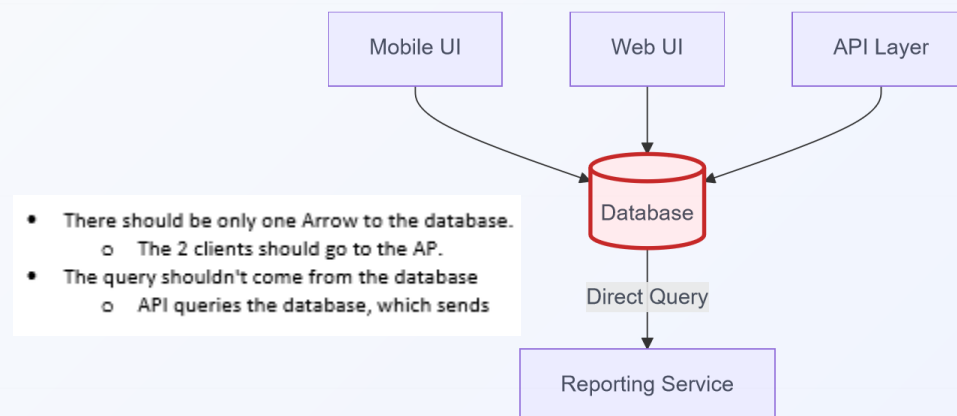
Recommendation for Your Projects:

Most projects should use **Client-Server** as the base pattern, potentially with **Layered** organization within the server.

Rule: Start simple. You can always refactor later.

Activity: Architecture Detective

Scenario: Review this flawed system design:



Your task (3 min):

- Find 2 problems
- Write 1 fix for each

Writing SMART Tasks

The SMART Framework:

Letter	Meaning	Question to Ask
S	Specific	What exactly will you do?
M	Measurable	How will you know it's done?
A	Achievable	Can you realistically finish this?
R	Relevant	Does this support the sprint goal?
T	Time-bound	When will you finish?

Task Evolution Example:

Version	Task Description
Bad	"Learn Flask"
Better	"Watch Flask tutorial"
Good	"Install Flask and test with hello world"
SMART	"Install Flask, create app.py with /health endpoint that returns JSON {'status': 'ok'}, test with curl. Due: Friday. Est: 2 hours."

Note: The SMART example above would be a Week 5 task after you learn Flask.

SMART Task Examples

Week 4 Tasks (What you CAN do now):

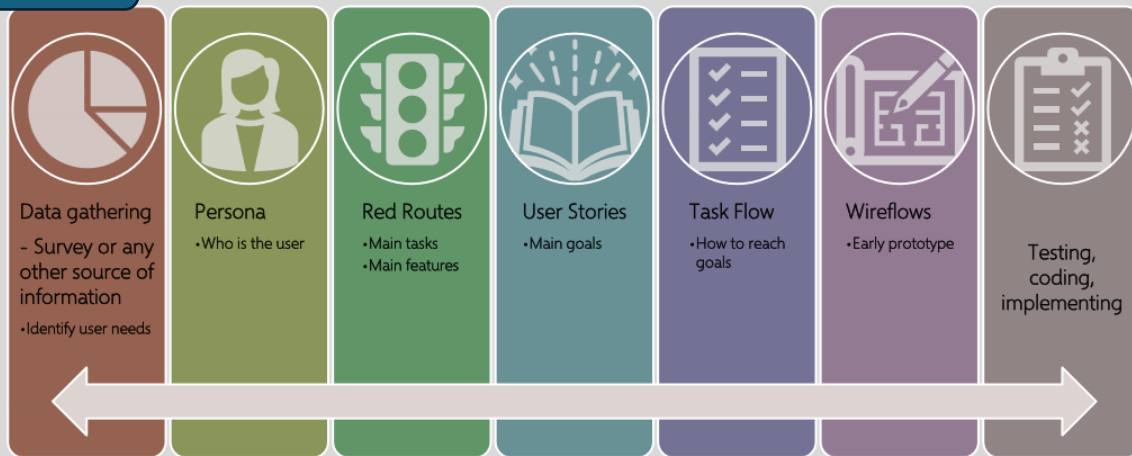
Task Description	Est.
Draw Context Diagram & add to docs/architecture.md	1h
Create all Sprint 1 tasks as GitHub Issues with labels and assignments	1h
Research and document 2 possible Flask starter projects/templates	1h
Identify data models (classes) needed for your project and create UML class diagram	2h

Week 5 Tasks (To add to your backlog):

Task Description	Est.
Install Flask and create hello world app	2h
Connect a simple to trigger Server logic and display JSON response.	2h
Setup database and test connection	2h

Total Estimated Hours for Sprint 1: ~8-10 hours per person

3.1



At any stage, you might need to revisit the previous step and modify things.
This is the nature of design thinking.

What if you picked several people and designed a pen to meet their specific needs?

Sara

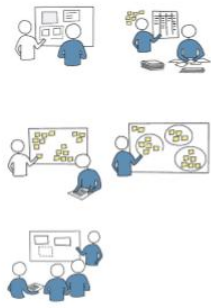


Philip



Kim





How to gather data for your UX project?

User Interviews

There's a huge difference in how users describe a product and how they would actually use it. **So user interviews are effective when they have something to interact with.**

Field Studies

Conducted in the user's natural environment. Observing a doctor work vs. letting them describe their work can be a much bigger value.

Diary Studies

Means asking users to log specific actions for a certain period. This creates a very detailed image of their behavior. Like how much time they talk with their parents on the phone.

Quantitative surveys

Measure numbers and metrics or compare 2 or more designs. These are close-ended questions.

Qualitative surveys

It generates unpredictable insights with open-ended questions. You can gather stories, experiences or personal examples.

How to make Persona?

- First **Research** your users
- Design for somebody who doesn't exist, and you'll have no customers.
- Identify patterns in your data
- Group them into clusters (e.g., Affinity Diagram)
- Create and prioritize user personas

What does Persona Do?

- Personas make assumptions about users explicit.
- Personas place the emphasis on specific users rather than "everyone".
- In limiting our choices, personas help us make better design decisions.
- Personas help the team gain a shared understanding of users all the way through.

Don't have too many personas

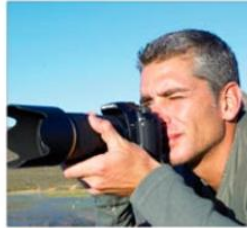
A small number of personas will make easier to trace design decisions.

Your preliminary Personas



Experienced Hikers:

- Want to discover new routes.
- Gradually choose harder one
- Keen to stay fit



Nature Lover:

- Like nature
- Wants to enjoy their time while hiking.



Active retirees

- Spend time with people and socialize
- Mainly like easy hikes
- Not very familiar with Technology

primary persona

Name & Sketch	Description
 John	• 35 years old • Lives in Vancouver, BC • Married with three kids • Working full time job • Environmental scientist
Behaviors	Needs & Goals
• Likes nature • Want to enjoy his free time while hiking • Takes photos of nature • Pick different hiking trails to explore variety in nature • Sometimes take his kids for a hike too • Hike during weekend or holidays	• Need a list of the hikes • Allow him to easily find and plan hikes that are appropriate for her and her family's skill level and interests. • Likes to see some pictures of the hike to get the sense of nature • Needs to see the hike map • Needs to know details about the hike, duration, length, height, etc • Would like to read reviews of people who tried this hike to get better idea

We produced three personas, but due to time and resource constraints, we are going to pick one as a **primary persona** and focus on that for this project.

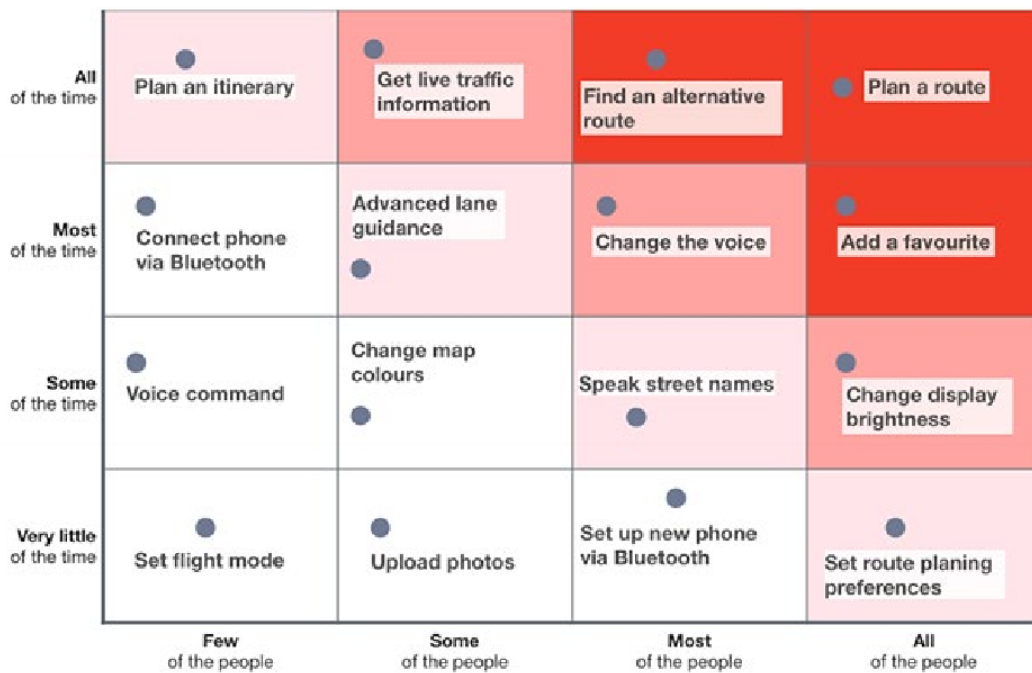


At any stage, you might need to revisit the previous step and modify things.
This is the nature of design thinking.

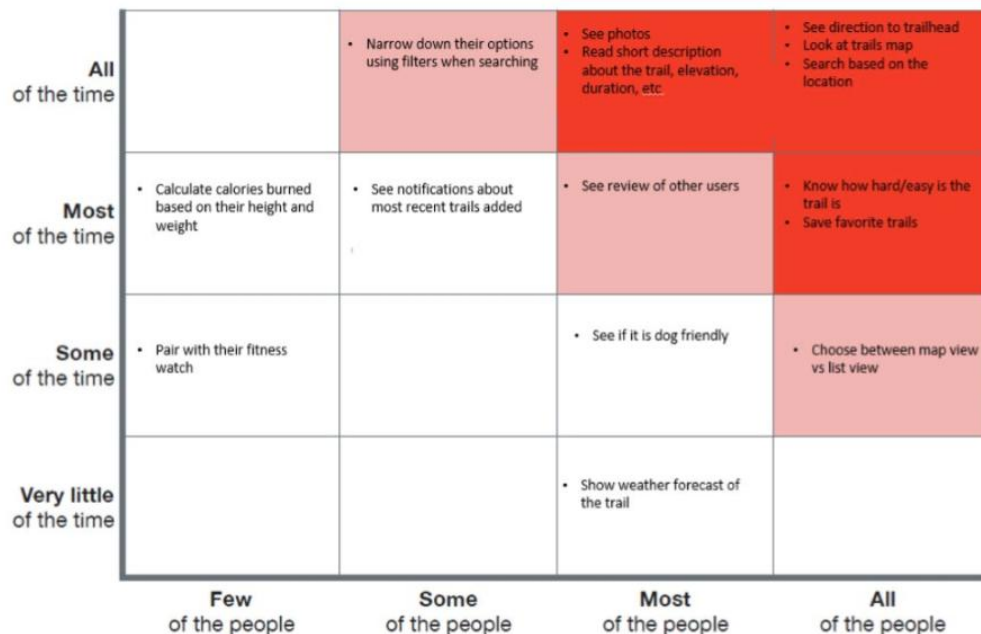
◦ Steve Jobs

"I know you have a thousand ideas for all the cool features iTunes could have. So do we. But we don't want a thousand features. That would be ugly. Innovation is not about saying 'Yes' to everything. It's about saying 'No' to all but the most crucial features."

Red routes of in-car satellite navigation system



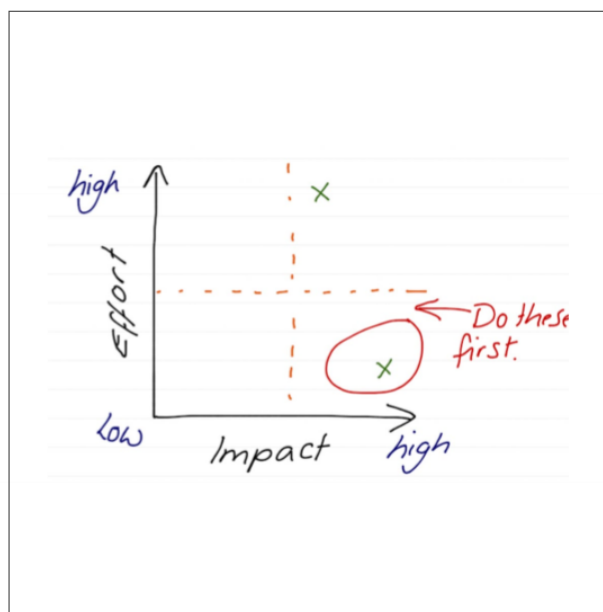
Our Hike App Red Routes:



Some questions to consider when designing a new feature:

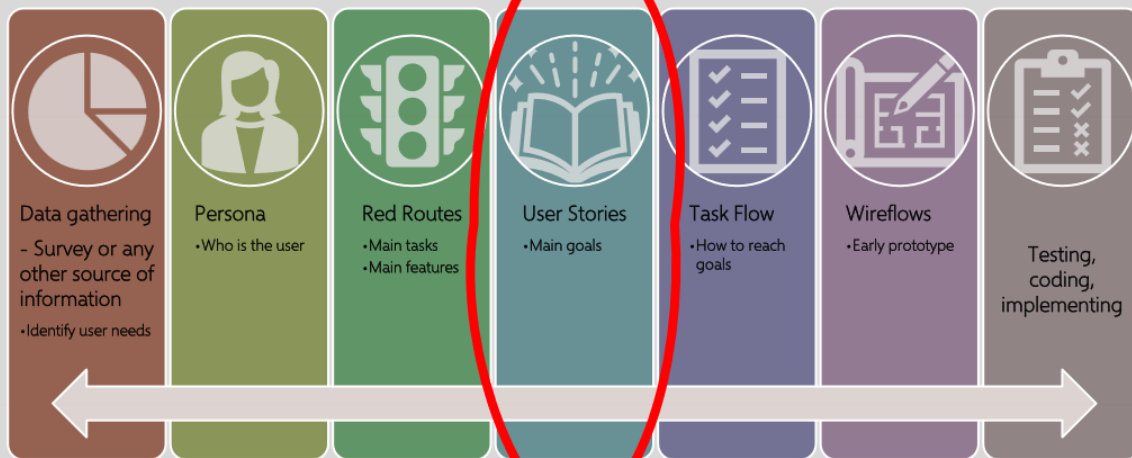
A distinct function

- What problem are you solving?
- Why is the user choosing to solve this? (triggers)
- How are you solving the problem?
- Is there an existing **pattern** for this solution?
- What is the situational context when interacting with this feature?
- How frequently will this feature be needed? (80/20 rule)
-

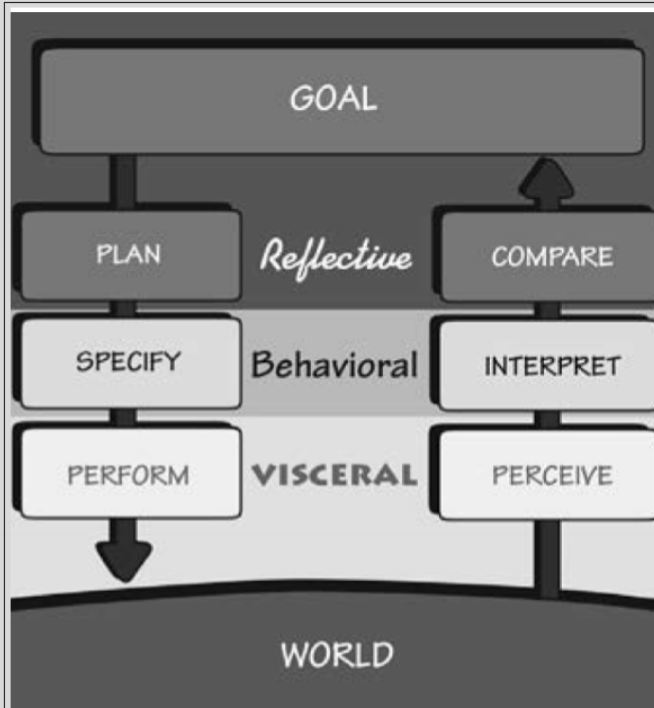


Feature Analysis

- Not all features are equally important.
- Impact = importance = urgency = priority
 - Example: the "create a group" feature must be done FIRST, before "show ranking in group" feature.
- Effort is estimated relatively to other features.



At any stage, you might need to revisit the previous step and modify things.
This is the nature of design thinking.



What is user story?

- User scenarios are short stories that tell us about our users' motivation, their goals and actions on our website.
- When writing a user scenario, keep it short and to the point.
- Good user scenarios help to
 - define concrete design requirements
 - improve our usability
 - achieve positive user experience.

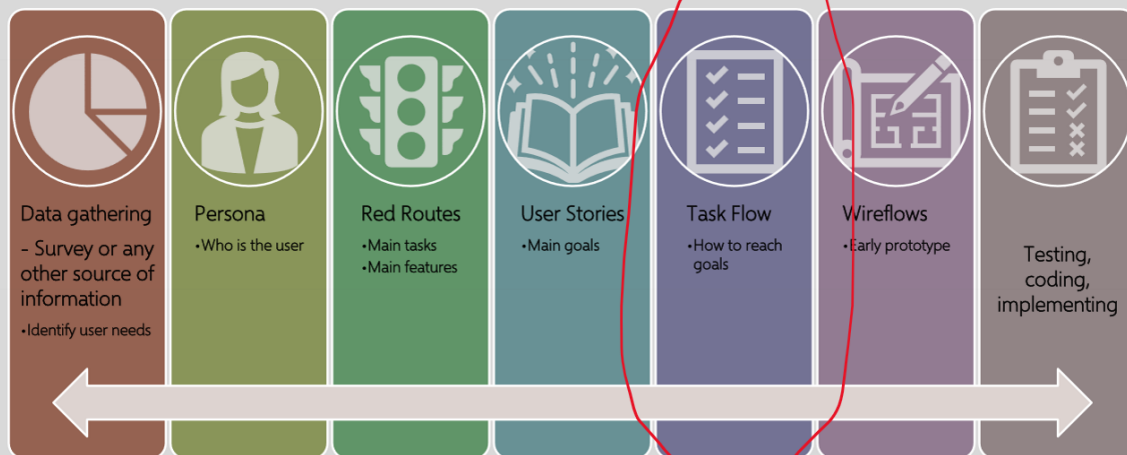
Example for hiking app:



Red routes:
Look at the trails map



User Story:
As a hiker,
I need to download the trail map,
so that I can look at it during hiking.



At any stage, you might need to revisit the previous step and modify things.
This is the nature of design thinking.

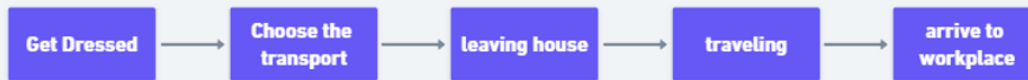
Task Flows are:

- *Linear*
- *Sequential*
- *Focus on main success scenario*

Example:

User Story: As an employee, I need to use transportation so that I can get to work.

Task - [commute to work](#)



Steps defined above could be different for different personas.

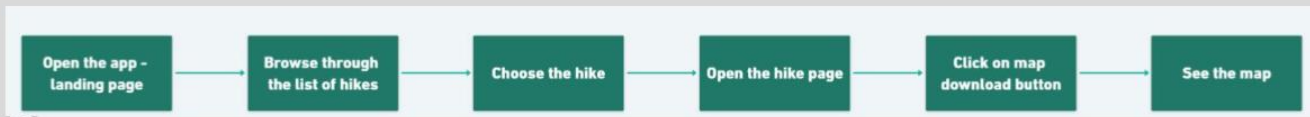
For example, someone might choose Skytrain while another person choose to drive.



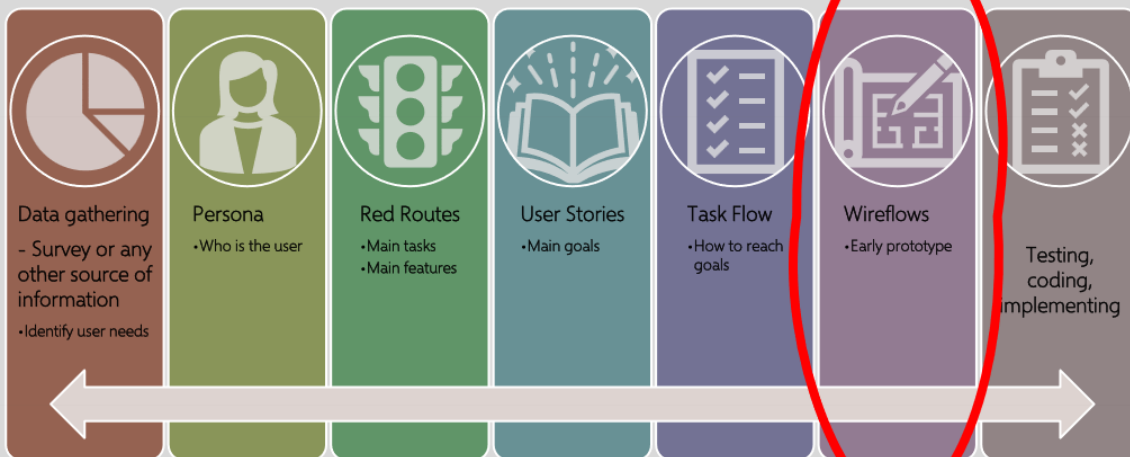
Task Flow is not a User Flow

- Task flows focus on single tasks (usually No decision point)
- A set or series of task flows can be embedded within *user flows*, but unlike task flows, user flows include *decision points* wherein the persona's journey to the desired goal can differ, based on decisions the persona needs to make in interacting with the design.

Hike App Example

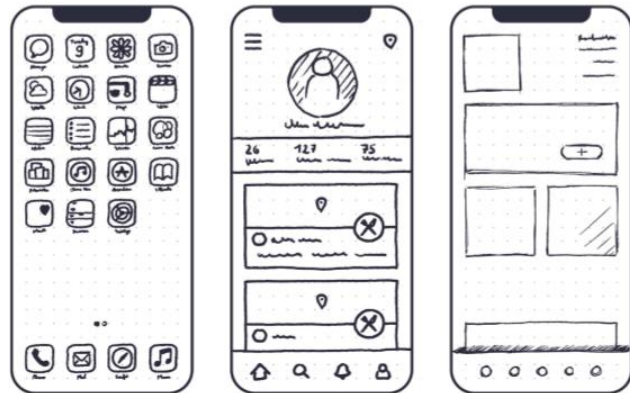


Notice that they all start with verbs because task flows are series of actions



At any stage, you might need to revisit the previous step and modify things.
This is the nature of design thinking.

WIREFLOW

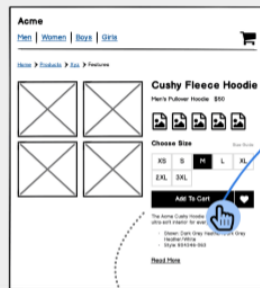


Wireflows = Wireframe + Task Flows

Wireflow is visualized version of task flow

Purchase Flow

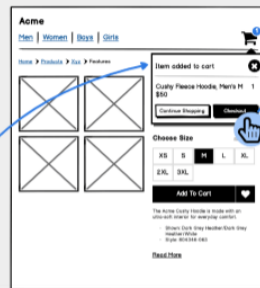
Product View - User Adds to Cart



If Size isn't Selected



Cart Preview



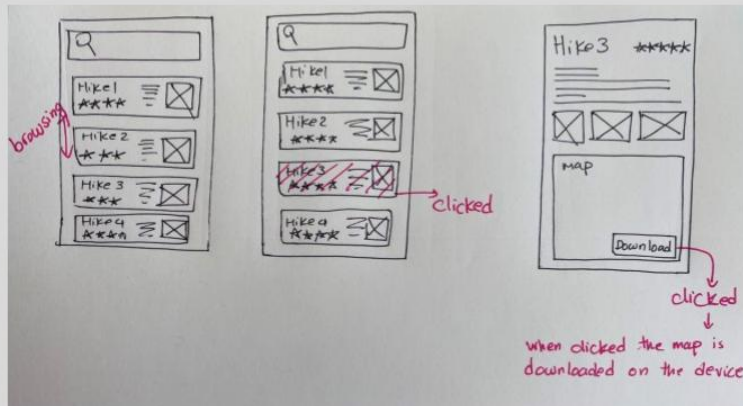
Checkout - Cart



Checkout - Contact Info

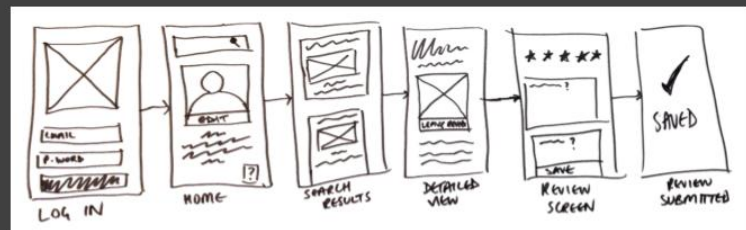


Hike App Example – Downloading the map



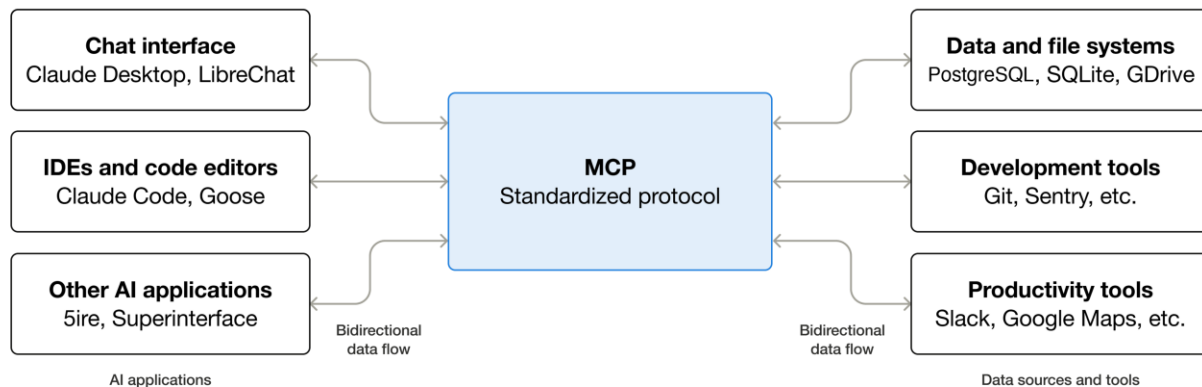
YOUR TURN

For the same task flow, draw a simple Wireflow using pen and paper.



What is the Model Context Protocol (MCP)?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems. Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks. Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.



What can MCP enable?

- Agents can access your Google Calendar and Notion, acting as a more personalized AI assistant.
- Claude Code can generate an entire web app using a Figma design.
- Enterprise chatbots can connect to multiple databases across an organization, empowering users to analyze data using chat.
- AI models can create 3D designs on Blender and print them out using a 3D printer.

Why does MCP matter?

Depending on where you sit in the ecosystem, MCP can have a range of benefits.

- Developers: MCP reduces development time and complexity when building, or integrating with, an AI application or agent.
- AI applications or agents: MCP provides access to an ecosystem of data sources, tools and apps which will enhance capabilities and improve the end-user experience.
- End-users: MCP results in more capable AI applications or agents which can access your data and take actions on your behalf when necessary.

What is a Database?



The Filing Cabinet Analogy

Think about how you organize files on your computer:

- **Folders** group related files together
- **Files** contain your actual data
- **File names** help you find things quickly

A database works the same way, but it's:

- **Persistent** - survives restarts and crashes
- **Searchable** - find data instantly
- **Shared** - multiple users can access it
- **Protected** - prevents data corruption

Why Not Just Use Files?

PLAIN FILES	DATABASE
Manual searching	Instant queries
Can corrupt if app crashes mid-write	Atomic transactions
Hard to share between servers	Built for multiple connections
You manage everything	Handles complexity for you

SQL Databases (Relational)

How It Works

- Data stored in **tables** with rows and columns
- You define a **schema** upfront (fixed structure)
- Tables can **relate** to each other (foreign keys)
- Query using **SQL** (Structured Query Language)

Best For

- Strong data consistency and integrity
- Complex relationships between data
- Financial systems, inventories, user accounts

Examples: PostgreSQL, MySQL, SQLite, SQL Server

Example: Sensors Table

ID	NAME	TYPE	LOCATION
sensor-001	Temp	temperature	Room 101
sensor-002	Humid	humidity	Room 102

Example Query (SQL)

```
-- Get one sensor
SELECT * FROM sensors WHERE id = 'sensor-001';
```

Key trait: Structured, predictable, great for complex queries and relationships.

NoSQL Databases (Document-Based)

How It Works

- Data stored as **documents** (JSON-like objects)
- **Flexible schema** — each document can differ
- Documents grouped into **collections**
- No complex joins; data often nested

Best For

- JSON-based REST APIs
- Rapid development with evolving data
- High-scale, distributed systems

Examples: MongoDB, DynamoDB, CouchDB, Firestore

Example: Sensors Collection

```
// Document: sensor-001
{
  "name": "Temp Sensor",
  "type": "temperature",
  "location": "Room 101"
}
```

Sample Firestore Query

```
db.collection('sensors').where('type', '=', 'temperature')
```

Key trait: Flexible, JSON-native, perfect fit for REST APIs like ours.

Why Firestore?

Firestore is an enterprise-grade, serverless NoSQL document database designed for mobile, web, and server development. Using a cloud DB means:

- Database calls are **network calls** (latency, failures, retries matter)
- Data is **centralized**, so restarts and “new server instances” don’t lose state

Why Firestore specifically?

- **No setup required** - runs in the cloud
- **JSON-native** - stores data exactly like our API payloads
- **Free tier** - good for learning
- **Scales automatically** - handles growth
- **Real-time capable** - for future projects

We chose Firestore because it matches how we already think about data (JSON objects) and requires zero database administration.

Firestore Mental Model: Folders and Files

Think About Your Computer

YOUR COMPUTER	FIRESTORE
Folder	Collection
File	Document
File name	Document ID
File contents	Fields (JSON data)

Example Firestore DB: Storing Sensors

```
sensors/                                <-- Collection
├── sensor-001/                          <-- Document ID
│   ├── name: "Temp Sensor"             <-- Field
│   ├── type: "temperature"             <-- Field
│   └── location: "Room 101"             <-- Field
└── sensor-002/
    ├── name: "Humidity"
    ├── type: "humidity"
    └── location: "Room 102"
```

That's it. Collections hold documents. Documents hold fields. Fields are just key-value pairs.

CRUD: The Four Database Operations

Every DB interaction falls into four categories

OPERATION	HTTP METHOD	WHAT IT DOES	EXAMPLE
Create	POST	Add new data	Add a new sensor
Read	GET	Retrieve data	Get sensor details
Update	PUT/PATCH	Modify existing data	Change sensor location
Delete	DELETE	Remove data	Remove a sensor

Our Plan

We'll implement each one:

- 1. **Create:** POST /api/sensors - Add a new sensor
- 2. **Read (one):** GET /api/sensors/<id> - Get one sensor
- 3. **Read (all):** GET /api/sensors - List all sensors
- 4. **Update:** PUT /api/sensors/<id> - Update a sensor
- 5. **Delete:** DELETE /api/sensors/<id> - Delete a sensor

CREATE: Adding a Document to Firestore

The Pattern

```
collection_ref = db.collection('sensors')
doc_ref = collection_ref.document('sensor_id')
doc_ref.set({
    'name': 'Temp Sensor',
    'type': 'temperature'
})
```

What `.set()` Does

- Creates the document if it doesn't exist
- **Overwrites** the document if it does exist
- Returns immediately (data is saved)

Full Endpoint Example

```
@app.route('/api/sensors', methods=['POST'])
def create_sensor():
    data = request.get_json()
    sensor_id = data.get('id')
    doc_ref = db.collection('sensors').document(sensor_id)
    doc_ref.set(data)
    return jsonify(
        {'message': 'Created', 'id': sensor_id}
    ), 201
```

READ: Getting Documents from Firestore

Get ONE Document

```
doc_ref = db.collection('sensors').document('sensor-001')
doc = doc_ref.get()
if doc.exists:
    data = doc.to_dict()
```

Get ALL Documents in a Collection

```
docs = db.collection('sensors').stream()
for doc in docs:
    print(doc.id, doc.to_dict())
```

Full Endpoint Examples

```
@app.route('/api/sensors/<sensor_id>', methods=['GET'])
def get_sensor(sensor_id):
    doc = db.collection('sensors').document(sensor_id).get()
    if not doc.exists:
        return jsonify({'error': 'Not found'}), 404

@app.route('/api/sensors', methods=['GET'])
def get_all_sensors():
    docs = db.collection('sensors').stream()
    sensors = [{ 'id': d.id, **d.to_dict() } for d in docs]
    return jsonify(sensors), 200
```

UPDATE: Modifying Existing Documents

Two Options

Option 1: `.set()` - Replace Everything

```
doc_ref.set({'name': 'New Name'})
# WARNING: This deletes other fields!
```

Option 2: `.update()` - Change Specific Fields

```
doc_ref.update({'location': 'New Location'})
# Only changes 'location', keeps everything else
```

When to Use Which?

- Use `.update()` for PATCH requests (partial updates)
- Use `.set()` for PUT requests (full replacement)
- Use `.set(data, merge=True)` for safe partial updates

Full Endpoint Example

```
@app.route('/api/sensors/<sensor_id>', methods=['PUT'])
def update_sensor(sensor_id):
    data = request.get_json()
    doc_ref = db.collection('sensors').document(sensor_id)
    if not doc_ref.get().exists:
        return jsonify({'error': 'Not found'}), 404
    doc_ref.update(data)
    return jsonify({'message': 'Updated'}), 200
```

DELETE: Removing Documents

The Pattern

```
doc_ref = db.collection('sensors').document('sensor-001')
doc_ref.delete()
```

That's it! The document is removed.

Important Notes

- Deleting a document that doesn't exist does NOT cause an error
- You might want to check if it exists first
- Deleted data is gone forever (unless you have backups)

Full Endpoint Example

```
@app.route('/api/sensors/<sensor_id>', methods=['DELETE'])
def delete_sensor(sensor_id):
    doc_ref = db.collection('sensors').document(sensor_id)
    if not doc_ref.get().exists:
        return jsonify({'error': 'Not found'}), 404
    doc_ref.delete()
    return jsonify({'message': 'Deleted'}), 200
```

Test with curl:

```
curl -X DELETE http://127.0.0.1:5000/api/sensors/sensor-001
```

Today's Roadmap

Context

Last week, we connected our API to Firestore. However, in a distributed system, open communication is dangerous communication.

The Sprint 2 Goal

Build a secure **End-to-End Flow**:

- Client sends JSON data.
- Server **validates** input.
- Server reads/writes securely.
- **No secrets committed to code.**

Hardening APIs

Today, we build defenses for distributed software:

1. **AuthN & AuthZ**: Who are you? What can you access?
2. **Token-Based Identity**: JWTs for users, API keys for services.
3. **Threat Modeling**: Anticipate attacks before they happen.
4. **Defensive Coding**: Trust boundaries, input validation, and cost limits.

Four Ways to Send Data to Your API

1. Query Parameters (?key=value)

Appended to the URL after ?. Used for filtering, searching, pagination.

```
# GET /api/users?role=admin&page=1
role = request.args.get("role")
page = request.args.get("page", 1)
```

2. Route Parameters (/users/<id>)

Embedded directly in the URL path. Used to identify specific resources.

```
# GET /api/users/123
@app.route("/api/users/<user_id>")
def get_user(user_id):
    # user_id = "123"
```

Use **query/route params** for GET requests (they're visible in URLs).
Use **form/JSON** for POST/PUT when sending sensitive or structured data.

3. Form Data

Key-value pairs from HTML forms. Used in web routes.

```
# POST /login (from HTML form)
email = request.form.get("email")
password = request.form.get("password")
```

4. JSON Payload (application/json)

Structured data in request body. The standard for modern APIs.

```
# POST /api/users
data = request.json
name = data.get("name")
```

Dual Interface: Web Routes vs. API Routes

Web Routes (Browser Users)

Routes that return **HTML pages** for human users in browsers.

- **Auth**: Session cookies, server-side rendering
- **Response**: `render_template("page.html")`
- **Example**: A student logging into a dashboard via web form

API Routes (Programmatic Access)

Routes that return **JSON** for apps, scripts, or IoT devices.

- **Auth**: Bearer tokens in headers, API keys
- **Response**: `jsonify({"data": value})`
- **Example**: A mobile app fetching user profile data

Why Both Exist

WEB ROUTE	API ROUTE
/login → HTML form	/api/login → JSON token
/signup → HTML form	/api/signup → JSON response
/profile → HTML page	/api/profile → JSON data

Pattern: Your Flask app often needs **both** interfaces for the same functionality. Web routes serve browsers; API routes serve everything else.

Foundational Concepts: AuthN vs. AuthZ

Authentication and **Authorization** solve two different problems.

Authentication (AuthN)

- **Question**: "Who are you?"
- **Mechanism**: Passwords, Tokens (JWTs), API Keys.
- **Result**: The system verifies the identity of the client (User or IoT node).

Authorization (AuthZ)

- **Question**: "Are you allowed to do this?"
- **Mechanism**: Permissions, Roles (Role-based access control), Code Logic.
- **Result**: The system verifies the identity has rights to perform the requested action.

Rule of Thumb: Authentication happens at the **front door**. Authorization happens inside the **individual rooms**.

The Distributed Auth Problem & Tokens

The Old Way: Sessions

Historically, servers used "Sessions" in RAM to remember that a user was logged in. In modern cloud setups with multiple auto-scaling servers, sharing memory is too slow and complex.

The Modern Way: Tokens

Instead of the server remembering you, it gives you a digital ID card.

1. You send your password once.
2. The server hands back a **Token**.
3. You attach this token to every future request.

Introducing the JWT

The industry standard for this is the **JSON Web Token (JWT)**.

A JWT contains the user's ID and an expiration time. Crucially, it is **cryptographically signed** by the server.

Because of this signature, your Flask API can look at a JWT and instantly trust it *without* having to do a slow database lookup to ask, "Is this user logged in?"

Rule: Never trust client claims. Always verify the JWT's cryptographic signature server-side.

Linking Identity to Application Data

The Separation

The IdP handles *who* the user is (authentication), but your database handles *what* the user does (application data). How do we link them?

The Link Pattern

Use the IdP's unique user identifier (usually called `uid` or `sub`) as the primary key in your database.

- **Benefit 1:** Instant $O(1)$ lookups
- **Benefit 2:** Guaranteed uniqueness
- **Benefit 3:** Aligns with security rules

Example: Firebase Auth → Firestore

```
profiles/  
{uid}/  <-- Document ID matches Firebase Auth UID  
├ email: "user@example.com"  
├ role: "user"  
└ created_at: Timestamp
```

Identity Providers (IdP)

The Problem

Building secure authentication from scratch is hard: password hashing, token signing, session management, MFA, etc.

The Solution: Identity Providers

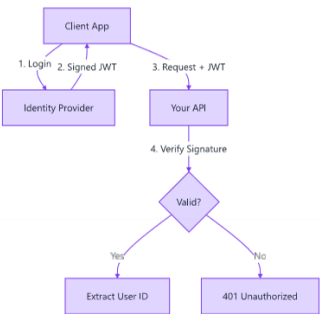
An **Identity Provider (IdP)** is a specialized service that handles authentication for you.

- **Popular IdPs:** Firebase Auth, Auth0, AWS Cognito, Okta
- **What they do:** Verify creds, issue JWTs, manage sessions
- **What you do:** Verify the JWT signature in your API

Separation of Concerns

- **IdP:** Manages user identities and issues tokens
- **Your Database:** Stores application data (profiles, settings)
- **Your API:** Enforces business logic and authorization

The Architecture



The Registration Flow

The "Two-Phase" Pattern

Creating a user requires two synchronized actions:

1. **Create Identity:** Register the user with your IdP
2. **Create Profile:** Initialize their application data using the returned user ID

Key Principles

- **Atomic Operation:** Both steps should succeed or fail together
- **Single Responsibility:** IdP manages auth, your API manages data
- **Validation First:** Always validate input before creating identity

Example: Web Route with Firebase

```
@app.route("/signup", methods=["GET", "POST"])  
def signup():  
    if request.method == "GET":  
        return render_template("signup.html")  
  
    # 0. Validate form input  
  
    # 1. Create identity in Firebase Auth  
    user = auth.create_user(  
        email=request.form.get("email"),  
        password=request.form.get("password")  
    )  
  
    # 2. Create profile in Firestore using uid  
    # 3. Redirect to appropriate page
```

The Login Flow

The Challenge

Your API server needs to authenticate users, but most IdP server-side SDKs are designed for *administration*, not logging users in.

Common Solutions

1. **Direct from Client:** The client app calls the IdP directly, receives a JWT, then sends it to your API
2. **Proxy via Server:** Your API accepts credentials, forwards them to the IdP, and returns the JWT

What You Get Back

A successful login returns a **signed JWT** containing:

- uid or sub: The unique user identifier
- exp: Expiration timestamp
- iat: Issued-at timestamp

Example: Firebase Identity Toolkit

```
WEB_API_KEY = os.environ.get("FIREBASE_WEB_API_KEY")

@app.route("/api/login", methods=["POST"])
def login():
    url = f"https://identitytoolkit.googleapis.com/v1/" \
        f"accounts:signInWithPassword?key={WEB_API_KEY}"

    res = requests.post(url, json={
        "email": request.json.get("email"),
        "password": request.json.get("password"),
        "returnSecureToken": True
    })

    if res.status_code == 200:
        return jsonify({"token": res.json()["idToken"]})
    return jsonify({"error": "Invalid credentials"}), 401
```

Protecting Routes with Decorators

What is a Decorator?

A decorator is a function that **wraps another function**. It runs code *before* your route executes.

- If the check fails, the decorator returns an error immediately
- If the check passes, the original route runs normally
- **Placement:** Custom decorators go directly under @app.route

Why Use Decorators for Auth?

Instead of repeating authentication logic in every route, decorators let you:

- Write the check once
- Apply it to any route with @require_jwt
- Keep route code clean and focused on business logic

The Pattern: JWT Decorator

```
from functools import wraps

def require_jwt(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        header = request.headers.get("Authorization")
        if not header or not header.startswith("Bearer "):
            return jsonify(
                {"error": "Invalid token format"}), 401

        token = header.split(" ")[1]
        try:
            decoded = auth.verify_id_token(token)
            return f(*args, uid=decoded["uid"], **kwargs)
        except Exception as e:
            return jsonify(
                {"error": f"Unauthorized: {str(e)}"}), 401
    return decorated
```

Verifying JWTs in Your API

The Pattern

Every protected route must verify the JWT before processing the request:

1. **Extract** the token from the Authorization header
2. **Verify** the cryptographic signature using the IdP's SDK
3. **Extract** the user ID to know *who* made the request

The Security Benefit

If the token is expired, tampered with, or invalid, verification fails safely blocking the request at the front door.

Example with Firebase Admin SDK

```
def verify_user():
    header = request.headers.get("Authorization")
    if not header or not header.startswith("Bearer "):
        return None, "Invalid token format"

    token = header.split(" ")[1]
    try:
        decoded = auth.verify_id_token(token)
        return decoded["uid"], None
    except Exception as e:
        return None, f"Unauthorized: {str(e)}"

@app.route("/api/profile")
def get_profile():
    uid, error = verify_user()
    if error:
        return jsonify({"error": error}), 401
    ...
```

Machine Identity: The API Key

Humans vs. Hardware

JWTs solve the human problem. But they are terrible for distributed hardware (e.g., remote microcontrollers):

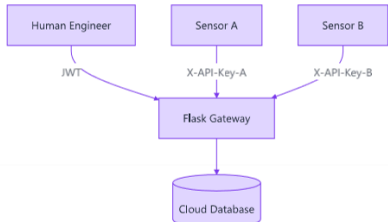
- JWTs expire frequently.
- Hardware struggles to request new tokens.
- No human operator to log in.

The API Key

We identify hardware using **API Keys**: secure, static strings injected directly into device firmware before deployment.

"Dual Identity" Architecture

Your API must route and verify human vs. machine requests differently.



Intro to Threat Modeling

What is Threat Modeling?

Security isn't just writing code; it's a mindset. Threat modeling is a systematic process to identify structural vulnerabilities *before* you build the system.

The 3-Step Framework

1. **Identify the Asset:** What holds value? (*User data, cloud billing, physical hardware*).
2. **Identify the Threat:** Who/what threatens it, and how? (*Hackers, stolen hardware, buggy code*).
3. **Design the Mitigation:** What architectural constraint stops it? (*Rate limits, data validation, encryption*).

Industry Context

This simplified framework draws from established methodologies like **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege)

Data Integrity & Trust Boundaries

The Trust Boundary Principle

Never trust data crossing the network boundary.

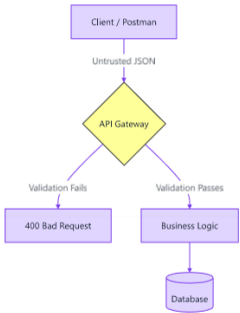
Client-side safety checks in hardware or apps mean nothing. Attackers can bypass clients entirely to send raw HTTP packets directly to the API.

Enforcing the Boundary

To protect your database, the server must actively verify:

1. **Presence:** Are required fields included?
2. **Type:** Is temperature actually a float?
3. **Range:** Is motor_speed between 0-100?

The Network Boundary



Example: Threat Modeling a Smart Lock

Let's apply the framework to a smart door lock connected to a cloud API.

1. The Asset

Physical access to a user's home and the safety of their family.

2. The Threat

An attacker sits outside with a Wi-Fi sniffer, captures the JSON payload { "action": "unlock" } sent by the owner's app, and "replays" that exact payload 10 minutes later.

3. The Mitigation

Require a single-use, time-sensitive "nonce" (number used once) in the payload. The server rejects old or repeated nonces, stopping the replay attack.

The EE Context: Assume Compromise

Unlike web servers locked in data centers, hardware sits out in the open (e.g., city traffic lights).

If an attacker physically steals a node, they can extract the API Key from its memory.

Rule: Always design APIs assuming the hardware *will* be physically compromised.

Defensive Validation Patterns

To enforce the Trust Boundary, implement these coding patterns in API endpoints:

1. Fail Fast

Check critical errors (missing auth, malformed JSON) immediately. Stop if found.

2. Collect All Errors

Check *all* fields before returning. Return a comprehensive error list to reduce round-trips.

3. Whitelist Fields

Only process fields you explicitly expect. Reject unknown fields.

4. Bounds & Pattern Checking

Validate length limits and format patterns (e.g., alphanumeric only).

Example: Validating a Profile Update

```
def validate_profile(data):
    errors = []
    allowed = {"first_name", "last_name", "student_id"}

    # 1. Whitelist Check - reject unknown fields
    if unknown := set(data.keys()) - allowed:
        errors.append(f"Unknown fields: {unknown}")

    # 2. Bounds Checking - enforce length limits
    if len(data.get("first_name", "")) > 50:
        errors.append("first_name must be 50 chars or less")

    # 3. Pattern Validation - enforce format rules
    sid = data.get("student_id", "")
    if not re.match(r"^[A-Za-z0-9]{8,9}$", sid):
        errors.append("student_id invalid")

    return errors  # Return ALL errors
```

Infrastructure: Denial of Wallet

Denial of Service (DoS)

Traditional DoS attacks overwhelm a server's CPU or RAM, causing it to crash.

Denial of Wallet (DoW)

Cloud databases (like Firestore) automatically scale to handle massive traffic. They won't crash, but **Google bills you for every read/write**.

A buggy while-loop on a remote sensor can rack up thousands of dollars in minutes.

Rate Limiting: The Token Bucket

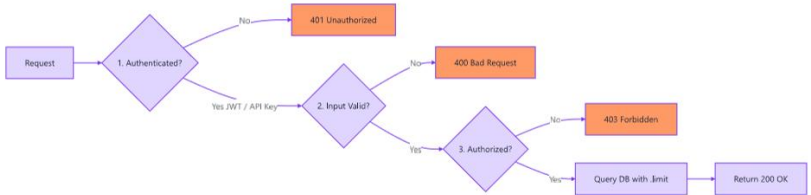
Cap requests per minute to prevent runaway loops:

1. Device is given a "bucket" of 100 tokens.
2. Every request costs 1 token.
3. Empty bucket = 429 Too Many Requests.
4. Bucket slowly refills.

The Secure Request Lifecycle

Integrated Pipeline

A well-architected API routes requests through strict checkpoints.



The Golden Rule: Fail Fast

Reject invalid requests at the earliest computational layer to minimize attack surface and save cloud costs.

Cost Protection: Safe Queries

The Database Threat

Open-ended querying allows attackers to pull millions of records in a single, incredibly expensive request.

Rules for Safe Queries

1. **Never** `get()` a full collection unconstrained.
2. **Force Pagination/Limits:** Set maximum returned records.
3. **Require Filters:** Demand a specific time range or device ID.

Vulnerable Code

```
# Fetches entire database!
docs = db.collection("logs").get()
```

Secured & Capped

```
# Capped at 50 records. Predictable cost.
docs = db.collection("logs") \
    .where("device_id", "==", req_id) \
    .order_by("timestamp", direction="DESC") \
    .limit(50) \
    .get()
```



Engineering Dashboards & Control Interfaces

COMP 7855 - Week 9

Hesam Alizadeh

Consumer UIs vs. Engineering Dashboards

Traditional Consumer Web

- **Focus:** Aesthetics, Engagement, SEO.
- **Interaction:** Occasional clicks, form submissions, page navigation.
- **Data Velocity:** Low (reading an article, loading a profile).

Engineering Dashboards

- **Focus:** Information density, immediate clarity, accuracy.
- **Interaction:** Continuous monitoring, emergency overrides, parameter tuning.
- **Data Velocity:** High (sub-second telemetry updates).

The "Alarm Fatigue" Problem

In engineering interfaces, if everything is highlighted, nothing is highlighted.

- **Rule of thumb:** A healthy system should look boring (e.g., neutral colors).
- Save bright red/flashing UI elements *strictly* for critical anomalies.

Evolution of Web Rendering

1. The Static Era

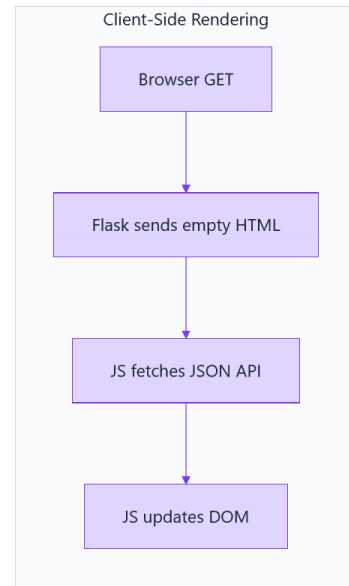
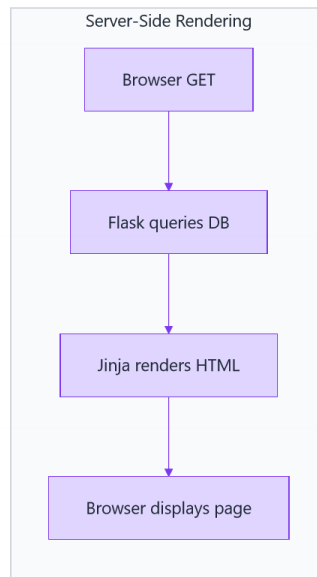
Server sends raw HTML. User clicks a link, the screen goes white, the server sends a completely new HTML file.

2. Server-Side Rendering (SSR)

Languages like PHP or Python dynamically stitch data into HTML templates *on the server* before sending it to the client. Requires full page reloads.

3. Client-Side Rendering (CSR)

The server sends an empty HTML shell. JavaScript in the browser fetches raw JSON data and draws the UI dynamically (React, Vue, Vanilla JS).



Activity 1: SSR vs. CSR Decision

The Scenario

You are building a dashboard for a **chemical mixing plant**.

- **Requirement A:** Operators must see temperature changes within 1 second.
- **Requirement B:** The page must be indexable by internal search tools for compliance logs.
- **Requirement C:** The network connection to the plant floor is intermittent.

Discussion

1. Which rendering paradigm (SSR or CSR) better fits Requirement A?
2. How does Requirement C (intermittent network) impact your choice of CSR?
3. Can you combine them? (Hint: Think about the initial load vs. live updates).

Deep Dive: Server-Side Rendering (SSR)

The server uses an HTML template, fills out the missing data, and serves a complete HTML document to the client.

Pros of SSR

- **SEO:** Search engines can read the fully formed HTML easily.
- **Initial Load Speed:** The browser gets a ready-to-display page instantly.
- **Security:** The data is processed securely on the server; the browser only sees the final output.

Cons for Engineering

- To see new data, the human operator must manually refresh. This is unacceptable for a live control room.

SSR Code (Flask/Jinja)

```
@app.route("/dashboard")
def dashboard():
    # 1. Fetch data on server
    data = db.collection("sensors").get()

    # 2. Build HTML on server
    return render_template(
        "dashboard.html",
        sensors=data
    )
```

```
<!-- Jinja injects data BEFORE sending -->
<ul>
    {% for s in sensors %}
        <li>{{ s.temp }}°C</li>
    {% endfor %}
</ul>
```

Deep Dive: Client-Side Rendering (CSR)

The server provides a static HTML file with placeholders. JavaScript running in the client browser continuously makes `fetch()` requests to your API and updates the screen.

Pros of CSR

- **Real-Time:** Can update parts of the screen without refreshing the page.
- **Decoupling:** Your backend API doesn't care about HTML. It just serves JSON. Mobile apps and Web apps can use the exact same API.

Cons for Engineering

- You must manage API tokens and state entirely in the browser's memory.

CSR Code (Fetch/JS)

```
// Client-side code running in browser
async function updateUI() {
    // 1. Ask API for JSON
    const res = await fetch('/api/sensors');
    const data = await res.json();

    // 2. Manually manipulate HTML
    document.getElementById("t1").innerText =
        data[0].temp + "°C";
}

// Trigger: Run every 2 seconds
setInterval(updateUI, 2000);
```

Security: Identity in the Browser

HTTP is Stateless

Every time the browser requests an image, an HTML file, or an API endpoint, the server has no memory of the previous request.

To stay logged in, the browser must attach a "Proof of Identity" to **every single request**.

Two ways to do this:

1. **Sessions (Cookies):** Managed by the browser automatically.
Used heavily in SSR.
2. **Tokens (JWTs):** Managed by your JavaScript code manually.
Used in CSR.

The Security Divide

We choose between Cookies and Tokens based on two major web vulnerabilities:

- **XSS** (Cross-Site Scripting)
- **CSRF** (Cross-Site Request Forgery)

Auth Method 1: Session Cookies (SSR)

How Cookies Work

When you log in via Flask, it sends back a special header:

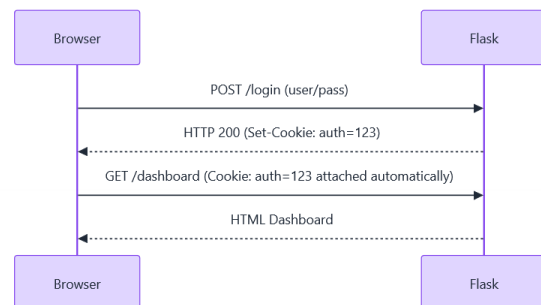
```
Set-Cookie: session=xyz
```

The browser stores this and automatically attaches it to all future requests.

Security Profile

- **Pro:** If marked `HttpOnly`, malicious JavaScript cannot read the cookie. This makes it immune to **XSS** (Cross-Site Scripting) attacks.
- **Con:** Vulnerable to **CSRF** (Cross-Site Request Forgery). If you visit `evil.com`, it can trick your browser into sending a request to your API, and your browser will automatically attach the cookie!

The Cookie Flow



Auth Method 2: Bearer Tokens (CSR)

How Tokens Work (Week 8 Recap)

When you log in via an API, the server returns a JSON Web Token (JWT) as a string. The browser does *not* save this automatically. Your JavaScript must store it (e.g., in a variable) and manually attach it as an `Authorization: Bearer <token>` header on every `fetch()` call.

Security Profile

- **Pro:** Immune to **CSRF**. `evil.com` cannot force your browser to attach the token, because the browser doesn't do it automatically.
- **Con:** Vulnerable to **XSS**. If a hacker injects malicious JavaScript into your site, they can read the token variable and steal it.

The Token Flow

```
// JS must manually store the token
let USER_TOKEN = "eyJhbGciOiJIUzI1Ni...";

async function getData() {
  // JS must manually SEND the token
  const response = await fetch('/api/data', {
    headers: {
      'Authorization': `Bearer ${USER_TOKEN}`
    }
  });
}
```

The Hybrid Architecture

Bridging the Gap

We use a hybrid approach to maximize security and usability.

1. **Initial Load (SSR):** The user logs in. Flask uses a secure **Session Cookie** to remember them and renders `dashboard.html`.
2. **The Handoff:** While Jinja is rendering the HTML on the server, it securely injects the user's JWT into a JS `<script>` tag.
3. **Live Updates (CSR):** When the browser finishes loading the HTML, JavaScript wakes up, grabs the injected token, and starts fetching live API data securely.

Writing Hybrid JS inside HTML

Put this at the bottom of `dashboard.html`:

```
<script>
  const TOKEN = "{{ current_user_jwt }}";

  async function fetchLive() {
    const res = await fetch('/api/data', {
      headers: {'Authorization': `Bearer ${TOKEN}`}
    });
    // ... update UI ...
  }

  // 3. Trigger it AFTER the DOM loads
  document.addEventListener("DOMContentLoaded", () => {
    fetchLive(); // Fetch immediately
    setInterval(fetchLive, 500); // Poll every 0.5s
  });
</script>
```

The Pipeline: The Real-Time Challenge

We are Secure. Now we need Speed.

Our hybrid frontend can securely talk to our API. But how do we handle continuous streams of telemetry?

The HTTP Limitation

HTTP was built for documents. The client asks, the server answers.

The server cannot initiate a conversation with the client. If a sensor detects a fire, the Flask server cannot push that alert to the browser.

Three Solutions for Dashboards:

1. **HTTP Polling:** JS asks every X seconds.
2. **WebSockets:** A persistent, two-way TCP pipe.
3. **MQTT:** The IoT industry standard publish/subscribe protocol.

Solution 1: HTTP Polling (Our Lab)

```
// Simple to write, but wastes bandwidth
// and cloud read/write costs.
setInterval(fetchTelemetry, 5000);
```

Analogy: Kids in the backseat asking "Are we there yet?" every 5 minutes. Most of the time, the answer is no (wasted request).

Solution 2: WebSockets

The Persistent Pipe

Instead of asking the server for data over and over, the browser opens a standard HTTP connection and says "Let's upgrade this to a WebSocket."

The connection stays open permanently. Both the client and the server can push data to each other instantly.

Why use WebSockets?

- **Ultra-low latency:** Perfect for real-time robotic controls or live charting.
- **Efficiency:** No wasted HTTP headers sent on every request.

The WebSocket Trigger (JS)

Unlike polling, WebSockets use an **event listener** that fires only when new data arrives.

```
// 1. Open the connection
const socket = new WebSocket('ws://api.myplant.com');

// 2. Trigger function ONLY when data arrives
socket.onmessage = function(event) {
  const data = JSON.parse(event.data);

  // Update dashboard instantly!
  document.getElementById("temp").innerText =
    data.temp;
};
```


Solution 3: The Industry Standard (MQTT)

What is MQTT?

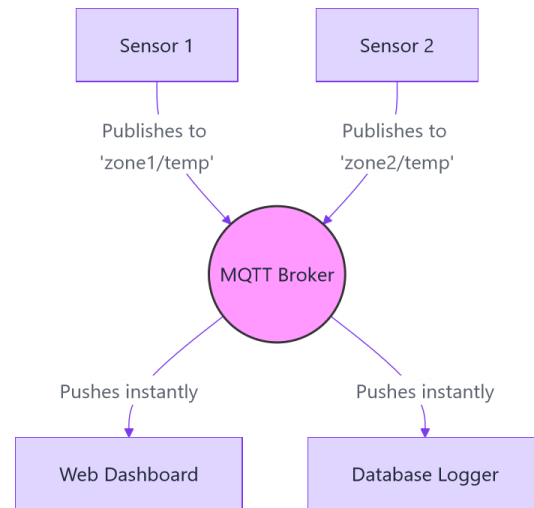
Message Queuing Telemetry Transport. It is a lightweight, low-bandwidth protocol specifically designed for IoT, SCADA, and distributed engineering hardware.

The Publish/Subscribe Model

Unlike HTTP or WebSockets (Client Server), MQTT uses a middleman called a **Broker**.

- **Clients don't talk to each other.** They talk to the Broker.
- Hardware **Publishes** data to a "Topic".
- Dashboards **Subscribe** to a "Topic".
- When data arrives, the Broker instantly pushes it to all subscribers.

MQTT Architecture



Visualizing Telemetry: Chart.js

The UI Challenge

Once data arrives via Polling, WebSockets, or MQTT, we have to render it. A table of 1,000 temperature readings is useless to a human operator.

Using Charting Libraries

We do not draw pixels manually. We use libraries like **Chart.js**, **Plotly**, or **D3.js**. Chart.js expects data in separate arrays for the X and Y axes.

Data Transformation

Your API returns an array of JSON objects. JavaScript must `.map()` this into arrays of labels and values before giving it to the chart.

The Transformation

```
// 1. Raw JSON arriving from your API
const data = [
  { time: "10:00", temp: 22 },
  { time: "10:05", temp: 25 }
];

// 2. Map to Arrays
const labels = data.map(item => item.time);
const values = data.map(item => item.temp);

// 3. Inject into Chart.js
myChart.data.labels = labels;
myChart.data.datasets[0].data = values;
myChart.update(); // Visually renders!
```

Controls: Closing the Loop (Actuation)

Observation vs. Actuation

- **Observation (GET):** Reading the temperature chart. Safe, repeatable.
- **Actuation (POST/PUT):** Turning on the cooling fan. **Alters the physical state of the system.**

Engineering Responsibility

Unlike a "Like" button on social media, a dashboard button moves physical matter.

- **Safety Critical:** A bug here doesn't just crash a page; it can damage hardware or injure operators.
- **Verification:** You must confirm the physical state matches the UI state.

The JS Control Trigger

How do we trigger an actuation? We bind it to an HTML element.

```
<!-- HTML UI Element -->
<button onclick="toggleMotor('ON')">Start Motor</button>
```

```
// JS Function sending command to API
async function toggleMotor(targetState) {
  const res = await fetch('/api/control/motor', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${TOKEN}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ active: targetState })
  });
}
```

API Design: Idempotency

Reminder

An operation is idempotent if doing it once has the exact same effect as doing it multiple times.

Why It Matters

Networks drop packets. Users double-click impatiently.

- **NOT Idempotent:** `POST /api/motor/increase_speed` (Click 6 times = Speed +60).
- **Idempotent:** `PUT /api/motor/set_speed` (Click 6 times = Speed set to 50, six times).

Designing Safe Controls

Always design control UIs to send **absolute target states**, never relative changes.

```
// BAD: State depends on history
// What if packets arrive out of order?
{ "action": "toggle_power" }

// GOOD: Idempotent target state
// Safe to send 100 times in a row.
{ "target_power": "ON" }
```

UI/UX: Optimistic vs. Acknowledged State

The Feedback Delay

It takes time for an HTTP request to travel, the database to update, and the physical motor to spin up. How should the UI button react when clicked?

1. Optimistic UI (Consumer Web)

Assume the server will succeed. Turn the UI button green instantly. (Used in apps like Instagram for "Liking" a photo to feel fast).

2. Acknowledged State (Engineering)

Never lie about physical state.

1. Click → Button turns Yellow (Pending).
2. Wait for API to confirm the database updated.
3. Turn button Green (Confirmed).

Connection to Idempotency

Why do we need "Pending" states? Because networks lag, users often double-click.

- Your UI "Pending" state protects the user experience, but **Idempotency** protects the physical system if the user bypasses the UI and clicks twice.
- **UX Rule:** If a physical action has real-world consequences, the UI must explicitly show the difference between "Command Sent" and "Action Verified".

System Architecture: The Monolith Problem

The Monolith Trap

Starting with a single `app.py` file is fast, but as a system scales, it becomes a bottleneck:

- **Hidden Coupling:** Changes in one area (e.g., user profiles) can unintentionally affect others.
- **Team Friction:** Multiple engineers editing the same files leads to frequent merge conflicts.
- **Poor Maintainability:** Large files increase the time required to understand and modify code.

Separation of Concerns (SoC)

The solution is **Modularity**: divide the application into isolated, self-contained domains.

Tightly Coupled (Monolith)

Changes in one part of the system can impact many others.

- Unexpected regressions
- Slower collaboration

Loosely Coupled (Modular)

Components are isolated and interact through clear boundaries.

- Safer, contained changes
- Easier parallel development

Flask Blueprints: Modularity in Practice

What is a Blueprint?

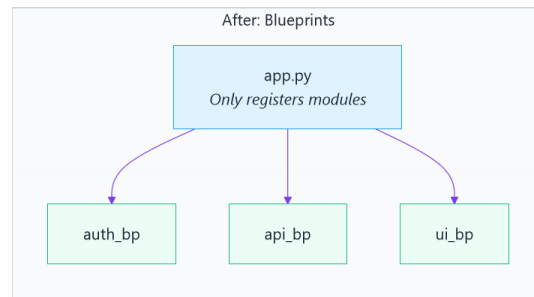
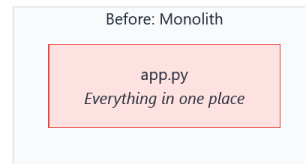
A **self-contained module** that groups related routes, templates, and static files.

Think of it as a "*mini-app*" that plugs into your main application.

The Pattern

```
app.py (orchestrator)
├── auth_bp  → /login, /logout, /register
├── api_bp   → /api/sensors, /api/actuators
└── ui_bp   → /dashboard, /settings
```

Before vs. After



Summary: The Full UI to Hardware Loop

Putting it all together

- 1. Identity Setup:** User logs into Flask. Flask creates a Session, renders the Dashboard (SSR), and injects the JWT into the JS `<script>` tag.
- 2. Data Fetching (Read):** JS uses the JWT to HTTP Poll (or open a WebSocket) to the Flask API securely.
- 3. Visualization:** JS transforms the incoming JSON into arrays and passes them to Chart.js.
- 4. Control Action (Write):** User clicks an HTML button which triggers a JS function. JS sends an idempotent target state (POST) to the API.
- 5. Actuation:** API updates the database, physical hardware reads it and adjusts.

Next Steps

In this week's Lab you will refactor backend using Blueprints, build the hybrid dashboard, wire up Chart.js, and actuate your system!

What is Software Testing?

Software testing is a process in which you execute your program using data that simulates user inputs and observe its behavior.

- **Tests pass** if the behavior is what you expect
- **Tests fail** if the behavior differs from what is expected
- If inputs are **representative** of a larger set, you can infer the program behaves correctly for all members of that set

Two causes of program bugs:

- **Programming errors:** Accidental faults in your code (e.g., off-by-one errors, wrong variable)
- **Understanding errors:** You misunderstood or were unaware of some requirement (e.g., didn't handle malformed data)

Testing can reveal the *presence* of bugs, but never prove their *absence*.

Your System's Architecture: Where Things Break

For the system we are building in our lab:

LAYER	WHAT YOU BUILT	POTENTIAL FAILURE MODE
Web Forms	Login, signup, profile pages	Missing fields, invalid input, session loss
Firebase Identity Toolkit	REST API call for login/signup	Bad credentials, network timeout, API key wrong
Firebase Admin SDK	<code>auth.verify_id_token()</code> in decorators	Token expired, tampered, clock skew
Firestore	Profile CRUD, sensor data storage	Permission denied, doc not found, quota exceeded
API Key Auth	<code>require_api_key</code> for device endpoints	Missing header, wrong key, key not configured
HTTP Polling	Chart.js fetches <code>/api/sensor_data</code> every 5s	Stale data, network drop, malformed JSON

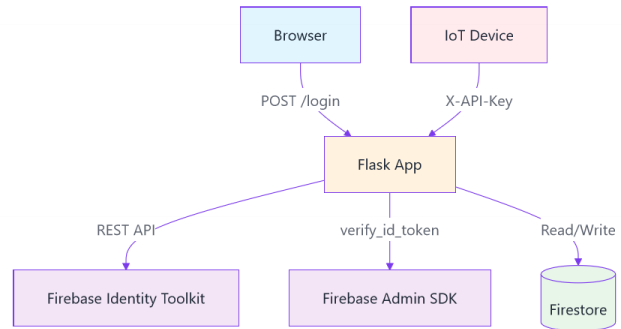
Distributed Systems Fail Differently

Single-Process Failures:

- Deterministic bugs
- Local state corruption
- Synchronous error propagation
- Easier to reproduce and debug

Your System's Distributed Failures:

- Firebase Auth is down → all users locked out
- Firestore write succeeds but read returns stale data
- JWT token expires mid-session → polling returns 401
- API key valid but Firestore quota exceeded → 500



Each external service is outside your control. Testing must account for their failures.

Why Mocking is Essential

The Problem:

Your code depends on external services you don't control.

Without mocking, your tests:

- Require real credentials and network access
- Are slow (100ms+ per network call vs. 1ms for a mock)
- Are non-deterministic (fail if Firebase is down)
- Leave dirty test data in your production database
- Cannot simulate rare errors (e.g., quota exceeded)
- Cost money (Firestore reads/writes accumulate)

The Solution: Mocking

Replace real services with controlled fakes.

With mocking:

- Run offline in milliseconds
- Produce identical results every time
- Simulate any error (timeouts, 403s, malformed responses)
- Run safely in CI/CD without secrets
- Test edge cases that are hard to trigger in production

Types of Software Testing

TEST TYPE	GOAL
Functional testing	Verify the system does what it's supposed to do. Discover bugs and provide evidence the system is fit for purpose.
User testing	Verify that the software is useful to and usable by end-users. Show that users can access features and use them effectively.
Performance & load testing	Verify the software responds quickly and handles expected user volume. Demonstrate it scales gracefully as load increases.
Security testing	Verify the software maintains integrity and protects user data from theft and damage.

Today we focus on **functional testing** (i.e., the staged process of verifying your code works correctly at every level.)

The Functional Testing Process

Functional testing is a **staged activity**. You start small and build up.

1. Unit Testing

- Test individual functions in isolation. e.g., `validate_profile_data()`

2. Feature Testing

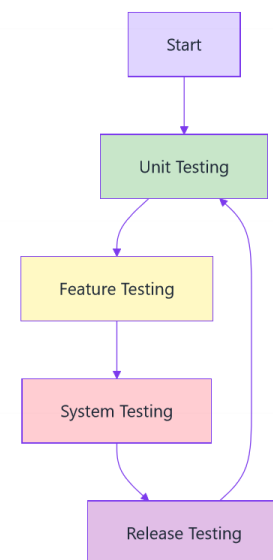
- Integrate units to test a complete feature
- e.g., Profile update = validation + normalization + Firestore write

3. System Testing

- Test interactions between features
- e.g., Login → Profile → Sensor data → Chart display

4. Release Testing

- Test in real operational environment
- Decide: Is the system good enough to deploy?



The Testing Pyramid

End-to-End Tests (~10%)

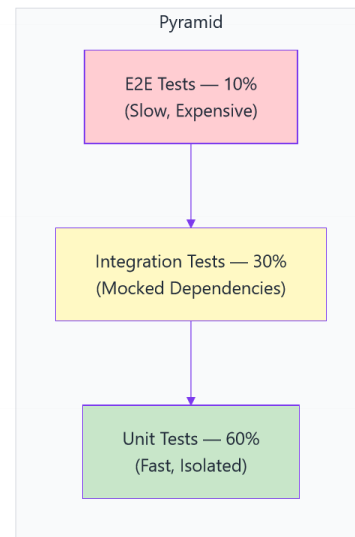
- Full system: browser → Flask → Firebase → Firestore
- Slow, expensive, brittle
- Hard to automate (GUI interactions)

Integration Tests (~30%)

- API endpoint + mocked Firebase + mocked Firestore
- Moderate speed
- Test component contracts

Unit Tests (~60%)

- Pure functions: validators, normalizers, helpers
- Fast, isolated, deterministic
- High code coverage target



Risk-Based Testing Priorities

Not all code deserves equal testing attention. Prioritize by:

Criticality (High Impact if Failed):

- `require_jwt` decorator → bypassing this = unauthorized access to all user data
- `require_api_key` decorator → bypassing this = unauthorized device data injection
- Firestore writes → data corruption or loss

Complexity (High Likelihood of Bugs):

- `api_update_profile()` → field whitelisting, validation rules, error accumulation
- Input validation with regex (`student_id` format checking)
- Edge cases in `normalize_profile_data()` (None values, whitespace)

Volatility (Frequently Changes):

- Validation rules (field length limits, allowed characters)
- New API endpoints added each lab
- Integration points with Firebase (SDK version changes)

Risk Score = Likelihood of Failure × Impact of Failure

Test Automation

Why automate?

Manual testing is boring, error-prone, and doesn't scale. You need hundreds of tests for a real application.

An automated test contains three things:

1. The **input data** to the unit being tested
2. The **expected result**
3. A **check** that the unit returns the expected result

You develop a set of executable tests and run them **every time you change the system**. If a test that previously passed now fails, you know your change broke something.

Automation infrastructure:

COMPONENT	ROLE
Test files	Collections of executable test functions
Test runner	Discovers and executes all tests (e.g., <code>pytest</code>)
Testing framework	Provides assertions, fixtures, reporting
Test report	Shows which tests passed, failed, and why

Test-Driven Development (TDD)

What is TDD?

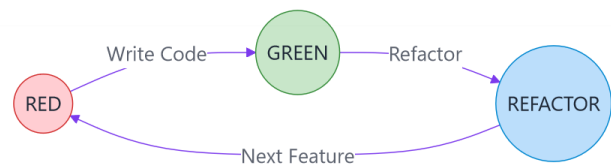
Write executable tests *before* you write the actual code.

The TDD Cycle:

1. **Red:** Write a test that fails (code doesn't exist yet)
2. **Green:** Write the simplest code to make it pass
3. **Refactor:** Improve without changing behavior

Benefits:

- Tests are clearly linked to code sections (high coverage guaranteed)
- Tests act as written specifications
- Debugging is simplified (failure = last change you made)
- Prevents over-engineering



Practical Limitation:

TDD works best for individual functions and API endpoints. It is more difficult to apply to full system testing or GUI testing.

TDD: Benefits and Limitations

ASPECT	EXPLANATION
✓ Systematic coverage	Tests are linked to code sections — no untested code ships
✓ Living specification	Tests document what the program does
✓ Simplified debugging	Failure → the last increment of code you added
✓ Simpler code	You only write code that's necessary to pass tests
X Discourages radical refactoring	Developers avoid changes that break many tests
X Focus on details over design	Developers may redefine the problem to make tests easier
X Implementation tunnel vision	Encourages focus on passing individual tests rather than overall program structure

Unit Testing: The Foundation

What is a Unit Test?

- Tests the smallest testable part of code — a function, method, or module with a single responsibility
- Isolated from all external dependencies
- Fast execution (milliseconds)
- Deterministic: same input → same output, every time

The core principle: If a program unit behaves correctly for a representative set of inputs, it will behave correctly for all inputs that share those characteristics.

What makes a good unit test? The FIRST principles:

PRINCIPLE	MEANING
F ast	Runs in milliseconds
I ndependent	No test depends on another test's result
R epeatable	Same result every time, in any environment
S elf-validating	Pass or fail (no manual interpretation needed)
T imely	Written before or at the same time as the code

The AAA Pattern

Every well-structured test follows three steps:

Arrange

- Set up test parameters and dummy data
- Create any mock objects needed
- Establish preconditions

Act

- Call the function being tested
- Pass in the arranged parameters
- Capture the result

Assert

- Verify the result matches expectations
- Check return values, side effects, or exceptions

```
# From YOUR codebase

def test_validate_profile_valid():
    # Arrange
    first = "Alice"
    last = "Smith"
    sid = "12345678"

    # Act
    error = validate_profile_data(
        first, last, sid
    )

    # Assert
    assert error is None
```

Equivalence Partitioning

You can't test every possible input. So how do you choose?

Equivalence Partitions group inputs that your code handles the same way. We only need to test one representative input.

You should identify partitions for **BOTH correct AND incorrect** inputs.

PARTITION TYPE	DESCRIPTION	EXAMPLE FOR STUDENT_ID
Valid 1	8 alphanumeric chars	"A1234567"
Valid 2	9 alphanumeric chars	"AB1234567"
Invalid 1	Too short (< 8 chars)	"ABC"
Invalid 2	Too long (> 9 chars)	"1234567890"
Invalid 3	Non-alphanumeric chars	"1234@678"
Invalid 4	Empty string	""
Invalid 5	None value	None

Boundary Testing

Once you've identified partitions, **test at the edges** where partitions meet. Bugs cluster at boundaries.

Guidelines for choosing test inputs:

GUIDELINE	EXPLANATION
Test edge cases	Choose inputs at the upper and lower bounds (lengths 7, 8, 9, 10 for student_id)
Force errors	Choose inputs that trigger every error message
Null and zero	Always test with <code>None</code> , empty strings <code>""</code> , empty lists <code>[]</code> , and <code>0</code>
One is different	If your code handles sequences, test with a single-element sequence
Keep count	After list transformations, verify the element count is consistent

Example: student_id boundaries:

- **Length 7** → Invalid (one below minimum), **Length 10** → Invalid (one above maximum)
- **Length 8** → Valid (at minimum), **Length 9** → Valid (at maximum),

pytest: A modern testing framework

pytest is the standard testing framework for Python. It eliminates boilerplate so you can focus on test logic.

Test Discovery Rules:

- Files named `test_*.py` or `*_test.py`
- Functions named `test_*`
- Classes named `Test*` with `test_*` methods

Powerful Assertions:

```
assert x == y           # equality
assert x is None        # identity
assert "error" in response  # membership
with pytest.raises(ValueError): # exception expected
```

Running Tests:

- `pytest` → run all discovered tests
- `pytest tests/test_validators.py` → specific file
- `pytest -k "test_validate"` → tests matching a pattern
- `pytest -v` → verbose output with each test name
- `pytest --cov=utils` → with coverage report

Parametrization

The Problem: Writing a separate test function for each equivalence partition creates massive code duplication.

The Solution:

`@pytest.mark.parametrize` runs one test function multiple times with different inputs.

Each parameter set:

- Represents one equivalence partition
- Runs as an independent test
- Reports pass/fail separately
- Makes failures easy to isolate

```
import pytest
from utils.validation import validate_profile_data

@pytest.mark.parametrize("first, last, sid, expected",
    [
        # Valid partition
        ("Alice", "Smith", "12345678", None),
        # Missing first_name
        ("", "Smith", "12345678", "All fields are required."),
        # Missing last_name
        ("Alice", "", "12345678", "All fields are required."),
        # Missing student_id
        ("Alice", "Smith", "", "All fields are required."),
        # None value
        (None, "Smith", "12345678", "All fields are required."),
        # All empty
        ("", "", "", "All fields are required."),
    ]
)
def test_validate(first, last, sid, expected):
    assert validate_profile_data(first, last, sid) == expected
```

Fixtures

Fixtures provide reusable test dependencies. Instead of repeating setup code in every test, you define it once and inject it.

Common fixtures for your system:

- Flask test client (simulates HTTP requests)
- Mock Firebase Auth (fakes token verification)
- Mock Firestore (fakes database operations)
- Sample test data (valid profile, invalid profile)

Fixtures are **injected by name** — if your test function has a parameter called `client`, pytest automatically provides the `client` fixture.

```
# conftest.py - shared across all test files
import pytest
from app import app

@pytest.fixture
def client():
    """Create a Flask test client."""
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

@pytest.fixture
def valid_profile():
    """Sample valid profile data."""
    return {
        "first_name": "Alice",
        "last_name": "Smith",
        "student_id": "12345678"
    }
```

Integration Testing: The Big Picture

Unit tests verify individual functions. But do your components work together?

Integration testing verifies the **interactions between components**:

- Flask route handler + auth decorator + validation logic + Firestore write
- Components may be developed by different people with different assumptions
- Bugs that unit tests miss often appear when units are combined

In your system, integration means testing:

- An HTTP request hits a Flask endpoint
- The auth decorator verifies the JWT
- The route handler validates input
- Firestore is called to read or write data
- A proper HTTP response is returned

Flask Test Client

What is it?

Flask provides a built-in test client that simulates HTTP requests without starting a real server.

What it gives you:

- Send GET, POST, PUT, DELETE requests
- Set headers (Authorization, Content-Type)
- Send JSON request bodies
- Inspect response status codes
- Parse response JSON

No network required. The test client calls your Flask routes directly in-process.

```
def test_profile_no_auth(client):
    """Missing auth header → 401."""
    response = client.get('/api/profile')

    assert response.status_code == 401
    data = response.get_json()
    assert "error" in data

def test_profile_bad_format(client):
    """Malformed auth header → 401."""
    response = client.get(
        '/api/profile',
        headers={
            'Authorization': 'NotBearer xyz'
        }
    )

    assert response.status_code == 401
```

These tests require no mocking. They test what happens BEFORE Firebase is even called.

Testing CRUD Operations

Your `/api/profile` endpoint supports full CRUD. Each operation needs both **success** and **failure** test cases.

OPERATION	TEST CASE	EXPECTED
GET	Valid JWT, profile exists	200 with profile data
GET	Valid JWT, no profile	200 with empty profile
GET	Missing JWT	401
POST	Valid JWT, valid data	200, profile saved
POST	Valid JWT, missing fields	400 with error
PUT	Valid JWT, valid partial update	200, profile updated
PUT	Valid JWT, unknown/invalid fields	400 with whitelist/format error
DELETE	Valid JWT	200, profile deleted
DELETE	Missing JWT	401

Core Mocking Concepts

Mock: A fake object that replaces a real dependency and records how it was used.

Patch: Temporarily replaces a real function/object with a mock during a test. Automatically restored after the test.

Key mock behaviors:

PROPERTY	WHAT IT DOES	EXAMPLE
<code>return_value</code>	What the mock returns when called	<code>mock.return_value = {"uid": "user1"}</code>
<code>side_effect</code>	Raise an exception or call a function	<code>mock.side_effect = Exception("expired")</code>
<code>assert_called_once()</code>	Verify the mock was called exactly once	Ensures your code actually called Firebase
<code>assert_called_with(arg)</code>	Verify what arguments were passed	Ensures correct token was passed to verify

Critical rule: Patch at the location where the function is **imported**, not where it's **defined**.

Mocking Firebase Auth

What you need to mock:

```
firebase_admin.auth.verify_id_token(token)
```

For success: Return a dict with `uid`

For failure: Raise an `Exception`

This is used inside your `require_jwt` decorator, which protects every API endpoint.

By mocking this one function, you can test all your API logic without a real Firebase project.

```
@pytest.fixture
def mock_firebase_auth(mockers):
    mock_verify = mockers.patch(
        'firebase_admin.auth.verify_id_token'
    )
    mock_verify.return_value = {
        "uid": "test_user_123"
    }
    return mock_verify

def test_get_profile_valid_jwt(
    client, mock_firebase_auth):
    response = client.get(
        '/api/profile',
        headers={'Authorization': 'Bearer any_token'})
    assert response.status_code == 200
    mock_firebase_auth.assert_called_once()
```

Mocking Firestore

The challenge: Firestore uses chained method calls.

Your code does:

```
db.collection("profiles")
  .document(uid)
  .get()
```

You need to mock the **entire chain**.

`MagicMock` handles this automatically! It creates nested mock objects for any attribute access or method call.

```
from unittest.mock import MagicMock

@pytest.fixture
def mock_firestore(mockers):
    mock_db = mockers.patch('firebase.db')

    # Create a mock document
    mock_doc = MagicMock()
    mock_doc.exists = True
    mock_doc.to_dict.return_value = {
        "first_name": "Alice",
        "last_name": "Smith",
        "student_id": "12345678"
    }

    mock_db.collection.return_value \
        .document.return_value \
        .get.return_value = mock_doc

    return mock_db
```


Simulating error conditions

Mocking isn't just for happy paths. Its real power is simulating failures you can't easily reproduce.

Firestore Auth errors:

```
def test_expired_token(client, mocker):
    mocker.patch(
        'firebase_admin.auth.verify_id_token',
        side_effect=Exception("Token expired")
    )
    response = client.get('/api/profile',
        headers={'Authorization': 'Bearer old_token'})
    assert response.status_code == 401
```

Firestore errors:

```
def test_firestore_permission_denied(client,
    mock_firebase_auth,
    mocker):
    mocker.patch('firebase.db',
        side_effect=Exception("Permission denied"))
    response = client.get('/api/profile',
        headers={'Authorization': 'Bearer token'})
    assert response.status_code == 500
```

Missing environment variable:

```
def test_api_key_not_configured(client, mocker):
    mocker.patch.dict(
        os.environ, {"SENSOR_API_KEY": ""})
    response = client.post('/api/sensor_data',
        json={"temp": 22},
        headers={"X-API-Key": "key"})
    assert response.status_code == 500
```

Feature Testing: Complete Workflows

Feature tests verify **complete user workflows** through your API by combining multiple integrated components.

An end-to-end pathway is a sequence of actions from starting a task through to completion.

Your System's Key Scenarios:

SCENARIO	COMPONENTS EXERCISED
Authenticated user retrieves profile	require_jwt → get_profile_data() → Firestore read → JSON response
User creates profile with valid data	require_jwt → validation → normalization → Firestore write → response
User updates profile with invalid field	require_jwt → whitelist check → error accumulation → 400 response
Unauthenticated request is rejected	require_jwt → exception → 401 response
Device submits sensor data	require_api_key → JSON validation → Firestore write → 201 response

Feature tests combine auth + validation + persistence in a single test. They catch integration bugs that unit tests miss.

Your Complete Testing Strategy

TEST TYPE	WHAT TO TEST	TOOLS	FROM YOUR CODEBASE
Unit	Pure functions	pytest + <code>@pytest.fixture</code>	<code>validate_profile_data()</code> , <code>normalize_profile_data()</code>
Integration	API + mocked Firebase	pytest-mock + Flask test client	<code>GET/POST/PUT/DELETE /api/profile</code>
Feature	End-to-end workflows	All of the above combined	Auth → Validation → Firestore → Response
Security	Auth enforcement	Flask test client	<code>require_jwt</code> , <code>require_api_key</code> decorators

Testing Workflow:

1. Write unit tests for validators and helpers (no mocking needed)
2. Add integration tests with mocked Firebase Auth and Firestore
3. Test all auth failure modes (missing token, invalid token, expired token)
4. Test all validation failure modes (missing fields, bad format, whitelist violations)
5. Run `pytest --cov` to verify coverage

Key Takeaways

- **Testing has a taxonomy:** Unit → Feature → System → Release
- **Equivalence partitions:** Group inputs by shared characteristics; test at boundaries
- **AAA pattern:** Every test follows Arrange, Act, Assert
- **TDD:** Write the failing test first to force clarity of intent
- **Test the pyramid:** Many fast unit tests, fewer slow integration tests
- **Partition your inputs:** Identify equivalence boundaries before writing tests
- **Mock external dependencies:** Firebase Auth, Firestore, and external APIs should never be called in tests
- **Automate everything:** Tests that aren't executable are useless
- **Security is testable:** Verify that `require_jwt` and `require_api_key` actually block unauthorized access



DevOps & Code Management

COMP 7855 - Week 13

Hesam Alizadeh

The Evolution of Software Support

The Traditional Silos

Historically, software delivery was divided across three isolated teams:

1. **Development:** Writes the code.
2. **Release/Operations:** Deploys the code.
3. **Support:** Deals with the angry customers.

The flaw: Developers had no incentive to write easily maintainable or deployable code.

The DevOps Shift

Amazon re-engineered their architecture so that each service was developed, released, and supported by the **same team**.

"You build it, you run it."

(The Amazon Principle)

DevOps merges Development and Operations. When the same engineers who write the code are responsible for keeping it alive in production, you get:

- **Faster deployment** (No waiting on other teams)
- **Faster repair** (The person fixing the bug actually wrote the code)
- **Greater accountability**

DevOps is a culture shift, not just a set of tools.

The Core Principles of DevOps

According to Ian Sommerville, true DevOps environments operate on three non-negotiable principles:

1. Everyone is responsible for everything.

No more "works on my machine" or "that's an IT problem." The team shares joint responsibility for development, delivery, and support.

2. Everything that can be automated, should be.

Manual deployment is a bottleneck and an error risk. Testing, deployment, and infrastructure setup must be scripted.

3. Measure first, change later.

Decisions must be driven by data collected from the live system, not by guessing.

The Business Benefits

Why do companies invest millions in DevOps transformations?

- **Reduced Risk:** Releasing small automated changes frequently is safer than releasing massive manual updates once a year.
- **Faster Problem Solving:** Small changes mean the source of a new bug is obvious.
- **More Productive Teams:** Eliminating manual, repetitive deployment tasks makes engineers happier.

Decoding CI/CD

The Code Management System (like GitHub) is the engine of DevOps. It triggers three levels of automation:

1. Continuous Integration (CI)

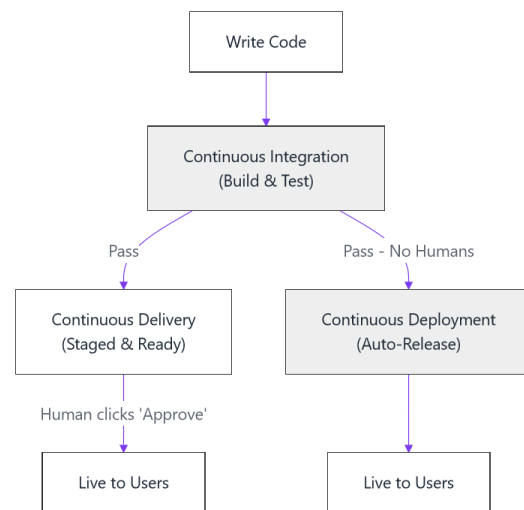
Every push to the `main` branch automatically triggers a build and a test suite. *(Goal: Catch bugs instantly).*

2. Continuous Delivery (CD)

Code that passes CI is automatically packaged and tested in a production-like staging environment. **A human must manually approve the final release.**

3. Continuous Deployment (CD)

Every change that passes automated tests is released directly to live users. **Zero human intervention.**



Continuous Deployment is only possible with total confidence in your automated tests.

More on CI/CD in wk 13

CI & "Breaking the Build"

The Cardinal Sin

"Breaking the build" means pushing code to the shared repository that causes the automated CI tests to fail.

When the build is broken, fixing it becomes the team's #1 priority.

The "Integrate Twice" Rule

To avoid breaking the build, developers must:

1. Integrate and test on their **local computer** first.
2. Push to the repo to trigger the **integration server**.

The Need for Speed

CI is only effective if the feedback loop is fast.

If developers have to wait 45 minutes for database populations or system compilations, they will stop using the process.

Solutions:

- Incremental building (only rebuild what changed — *like Docker layer caching!*)
- Mocking databases during tests.

Infrastructure as Code (IaC)

The Enterprise Problem

A large system might run on 50 web servers and 10 database servers. Manually installing software, configuring networks, and keeping them identical is impossible.

The DevOps Solution

Infrastructure as Code (IaC) defines the network, servers, and installed software as machine-readable models (code).

(e.g., Puppet, Chef, Terraform, Docker Compose).

Why IaC is Revolutionary

- **Reproducibility:** A script installs things in the exact same order every time. Testing and Production environments are identical.
- **Reliability:** Eliminates human typos during server configuration.
- **Recovery:** If a server is destroyed, you don't rebuild it manually. You run the script to generate an exact replica instantly.

Servers become disposable "cattle", not irreplaceable "pets".

DevOps Measurement

The DevOps Scorecard

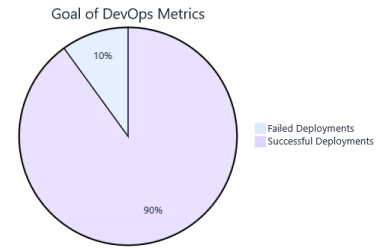
You cannot improve a process you do not measure. DevOps teams rely on specific metrics to prove their automation is working:

Process Metrics (How we work):

- *Deployment Frequency*: How often do we release to users? (Higher is better)
- *Percentage of Failed Deployments*: How often does a release cause a crash? (Lower is better)
- *Mean Time to Recovery (MTTR)*: When it breaks, how fast do we fix it? (Lower is better)

Service & Business Metrics (How the product performs):

- *Availability*: Is the system up? (Uptime %)
- *Customer Complaints / Usage*: Are users actually using the new features?



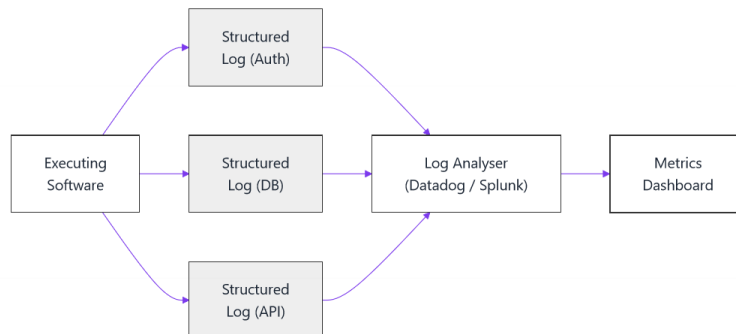
A healthy DevOps pipeline should constantly increase successful deployment frequency while driving MTTR (recovery time) toward zero.

Automating Measurement: Logging & Analysis

Instrumenting the Software

The DevOps principle of "automate everything" applies to measurement. You cannot have humans manually counting errors.

Your software must be **instrumented** to collect data about itself.



Without **Structured Logging**, log analyzers cannot parse the data, meaning your dashboards will be empty and your DevOps metrics will fail.

Summary: The DevOps Philosophy

- **Culture over Silos:** DevOps merges development, deployment, and support into one accountable team. *"You build it, you run it."*
- **Continuous Integration (CI):** Automates system building and testing on every commit to catch bugs instantly.
- **Continuous Deployment (CD):** Automates the release of software directly to customers, reducing the risk of massive, manual update failures.
- **Infrastructure as Code (IaC):** Replaces manual server configuration with version-controlled scripts, ensuring perfect reproducibility and instant recovery.
- **Measurement:** Driven by automated metrics (like Deployment Frequency and Mean Time to Recovery) gathered through structured logging.

DevOps is the ultimate application of software engineering principles to the infrastructure itself.



Infrastructure Security, Containers & CI/CD

COMP 7855 - Week 13

Hesam Alizadeh

The Feature Freeze

The Mindset Shift

You are no longer building features. You are making what you built **survivable in the real world.**

Managing Carry-Over Risk

If you try to build unfinished Sprint 3 features *while* learning Docker and CI/CD, your system will collapse under technical debt.

A small, well-deployed system earns a higher grade than a massive system that only runs on your laptop.

The New Rules

- No new features
- No new API endpoints
- No new Python libraries
- **Focus 100% on Infrastructure**

The 4 Pillars of Dependability

What makes a system trustworthy?

According to software engineering principles, a system is only dependable if it satisfies four dimensions:

- 1. **Availability** — Is the system up and able to be deployed anywhere? (*Docker / Containers*)
- 2. **Security** — Can it resist attacks at the network level? (*Compose / Isolation*)
- 3. **Safety** — If it fails, do we know why without causing harm? (*Structured Logging*)
- 4. **Reliability** — Does it behave correctly over time as code changes? (*CI/CD Quality Gates*)

The Goal
Move from: "It works on my laptop."
To: "It is formally proven to be Dependable."

Why Infrastructure Matters

INCIDENT	THE REAL-WORLD DISASTER	THE MISSING PILLAR	COST
Knight Capital (2012)	Deployed new code, but old code accidentally kept running alongside it.	Availability (Environment mismatch)	\$440M in 45 min
AWS S3 Outage (2017)	Engineer ran a maintenance command in the wrong environment.	Security (No environment isolation)	1000s of sites down
Equifax Breach (2017)	Known vulnerability went unpatched; intruders roamed undetected.	Safety / Security (No automated logging/CI)	\$1.4 Billion

"In almost every major catastrophic tech failure, the application code did exactly what it was programmed to do. The infrastructure is what failed."

Great code running on fragile infrastructure is worthless.

The Execution Environment

The Root Cause

Your software depends on a massive, hidden iceberg of configuration to run. We call this the **execution environment**.

When you share code via GitHub, you are sharing the tip of the iceberg. You are not sharing:

- Python version (3.9 vs 3.11)
- OS and system libraries (macOS vs Linux)
- Installed packages beyond `requirements.txt`
- Environment variables and file paths

The Gap

`requirements.txt` helps — but it doesn't capture:

- Which Python version to use
- Which OS the code was developed on
- What environment variables must be set
- Where credential files are located

When your teammate clones your repo, they get your code but not your environment. The same code produces different behavior on different machines.

The solution: make the environment itself a versioned artifact — buildable, shareable, reproducible.

From Virtual Machines to Containers

VMs (The Old Way)

Virtualizes the **hardware** via a hypervisor.

- Runs a full, heavy OS per app
- Uses **gigabytes** of RAM
- High maintenance overhead

Containers (The Modern Way)

Virtualizes the **OS** directly. Shares the host kernel.

- Packages only the app + dependencies
- Uses **megabytes** of RAM
- Takes **seconds** to start
- Dozens can run on one laptop

Three Key Traits

Immutable: Once built, the image doesn't change. What you tested is what you deploy.

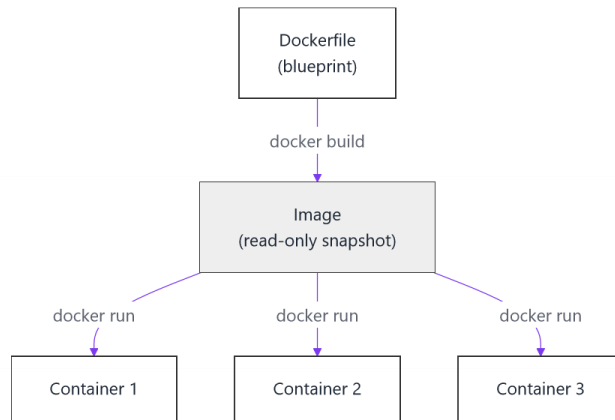
Isolated: Each container has its own filesystem, network, and process space.

Disposable: Destroy and recreate instantly. Containers are cattle, not pets.

Dependability Dimension 1: If it works in the container, it works everywhere.

The Docker Workflow

Docker has three phases: **build time**, **storage**, and **run time**.



Why This Matters

- **Dockerfile** — a text recipe defining your environment, line by line
- **Image** — an immutable snapshot. Once built, it never changes.
- **Container** — a running instance. Isolated, disposable, identical to every other container from the same image.

You write **one** Dockerfile, build **one** image, and run it anywhere — your laptop, a teammate's laptop, a CI server, or the cloud.

The Dockerfile and Layer Caching

```
# 1. Base Image (pinned version, slim variant)
FROM python:3.11-slim

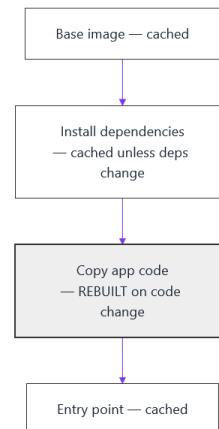
# 2. Working Directory
WORKDIR /app

# 3. Dependencies FIRST (changes rarely – cached)
COPY requirements.txt .
RUN pip install -r requirements.txt

# 4. App Code LAST (changes often – rebuilds)
COPY . .

# 5. Document the port and set entry point
EXPOSE 5000
CMD ["python", "app.py"]
```

Why Order Matters



Wrong order: every code change reinstalls all dependencies (minutes).

Right order: only the code layer rebuilds (seconds).

The Multi-Container Problem

Single vs. Multi-Container

A `Dockerfile` packages exactly **one** application.
But modern systems are distributed: an API, a Database, a Frontend.

Why not put them all in one container?

- Containers should do exactly *one* thing (separation of concerns).
- If the API crashes, the DB shouldn't go down.
- Scaling: You might need 3 API containers, but only 1 DB.

The Orchestration Challenge

When you split a system into multiple isolated containers, you must solve:

1. **Networking:** How does the API find the DB if they are isolated?
2. **Startup Order:** The API will crash if the DB isn't ready.
3. **Developer Experience:** Typing `docker run...` with 15 flags for 3 different containers every time you boot your laptop is a nightmare.

Coordinating multiple isolated containers to work as a single system is called **Orchestration**.

Orchestration: docker-compose

Infrastructure as Code

`docker-compose` defines your entire multi-container system in a single YAML file.

```
version: '3.8'
services:
  backend:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - database
    env_file:
      - .env
  database:
    image: postgres:15
```

What Compose Handles

- **Networking:** Automatically creates a secure internal network. `backend` can talk to `database` just by using its name as the URL.
- **Dependency Order:** `depends_on` ensures the DB starts first.
- **Simplicity:** One command to rule them all.

`docker-compose up --build`

(Builds images, creates networks, starts all services)

`docker-compose down`

(Tears the entire environment down cleanly)

Infrastructure Security: Isolation

Connecting to Week 8

In Week 8, you built **application-level** defenses:

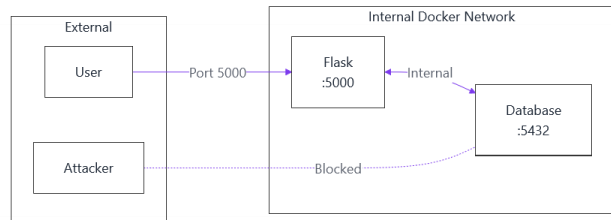
- AuthN (JWT at the front door)
- AuthZ (role checks in each room)
- Validation (trust boundary enforcement)

These live in your code. A missed route = attacker is through.

What Containers Add

Infrastructure-level defenses that work independently:

- Network isolation — databases invisible from outside
- Process isolation — compromised container cannot reach host



Dependability Dimension 4: Security. Week 8 built locks on the doors. Containers build the walls.

Secrets Management

Extending the rule: *no secrets committed to code.*

Containers: **no secrets baked into images.**

Images are portable and shareable. Anyone with the image can extract what's inside.

- Hardcoded in source code — anyone with repo access
- `ENV SECRET=...` in Dockerfile — extractable from image
- Committed to Git — persists in history forever

The Fix

1. Create a `.env` file with your secrets
2. Add `.env` to `.gitignore` immediately
3. Reference it in `docker-compose.yml`:

```
services:
  backend:
    build: .
    ports:
      - "5000:5000"
    env_file:
      - .env
```

The secret exists only at runtime, on the machine that needs it. It's never baked into an artifact.

A secret inside an image goes everywhere the image goes.

Headless Apps: Structured Logging

The Problem

Inside a container, your app is **headless**. No terminal. No breakpoints. If the container crashes and restarts, `print()` history is gone.

`docker-compose logs` is your only window.

Unstructured vs. Structured

```
something went wrong
```

vs.

```
10:32:10 [ERROR] Firestore write failed:  
user abc123, permission denied
```

Every entry needs: **when** (timestamp), **severity** (level), **what** (message with context).

What to Log

- Incoming requests and status codes (INFO)
- Auth success and failure (INFO / WARNING)
- Errors with context — who, what, why (ERROR)
- Startup configuration, non-secret (INFO)

What to Never Log

- Passwords or JWT tokens
- API keys or connection strings
- `.env` file contents

Same trust boundary from Week 8: your log output is a data boundary.

Dependability Dimension 3: Safety. If it fails, we must know exactly why.

Feedback Loops & Quality Gates

Continuous Integration (CI)

Automated system that intercepts every push, sets up a fresh environment, and runs tests.

Ensures the code is safe.

Continuous Deployment (CD)

Automated system that takes code that *passed* CI and pushes it to your live servers.

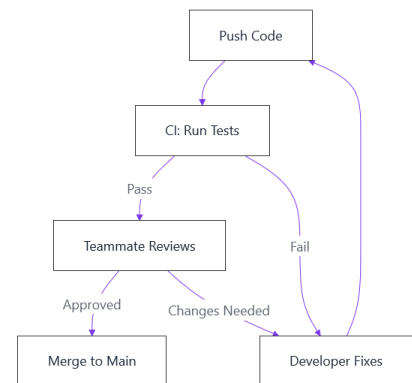
Ensures the code is shipped.

What is a Quality Gate?

A strict checkpoint that code must pass before it is allowed to merge.

- **Automated Gate (CI):** Catches functional regressions.
- **Human Gate (Code Review):** Catches bad architecture and logic flaws.

Dependability Dimension 2: Reliability. We automate tests so humans don't have to remember.



CI: The Clean Room

The Mental Model

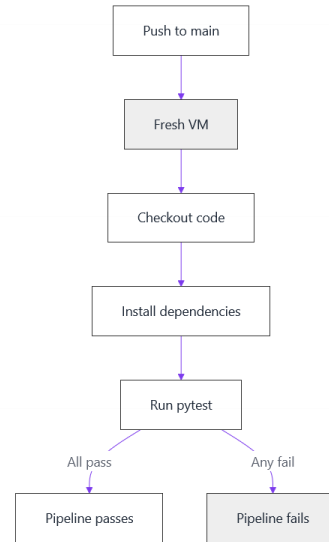
The CI runner starts **clean every time**. Nothing installed. No memory of previous runs.

Everything is set up from scratch by the workflow.

What This Means

- `requirements.txt` must be complete
- All tests must mock Firestore — there is no service account key
- Secrets are never committed — there is no `.env` file

If a test passes on your laptop but fails in CI, the test has a hidden dependency.



Conceptual Summary

- **The execution environment** — your code depends on a hidden iceberg of configuration. Containers make that entire environment a versioned, portable artifact.
- **Orchestration** — mapping multi-container systems securely using IaC (`docker-compose`).
- **Infrastructure security** — extends application defenses with network and process isolation. Independent layers that hold even when application code has bugs.
- **Secrets management** — no secrets in code, no secrets in images. Inject at runtime.
- **Structured logging** — the only window into a headless, containerized system. Timestamp, severity, context. Never log secrets.
- **CI/CD** — automated feedback loops. The clean room principle proves your code and tests are truly self-contained.

All facets of the same discipline: building systems that are trustworthy, inspectable, and handoff-ready.



Release Engineering & Continuous Delivery

COMP 7855

Hesam Alizadeh

The Final Gap: Local to Live

You have successfully implemented:

1. **Containerization (Docker):** a standard way to package environments and dependencies.
2. **Observability (Logging):** visibility into runtime behavior (what is happening and why).
3. **Automated Testing (CI Pipelines):** checks to verify correctness each time you change the code.

What is Missing

- How does tested code get from GitHub to a live server?
- How do we ensure that what we deploy matches what we tested —and remains consistent?
- How do we verify that the running system solves the original user problems, not just that tests passed?

The Three Phases of Automation

1. Continuous Integration (CI)

Frequently merge code and verify it with automated tests.

2. Continuous Delivery (CD)

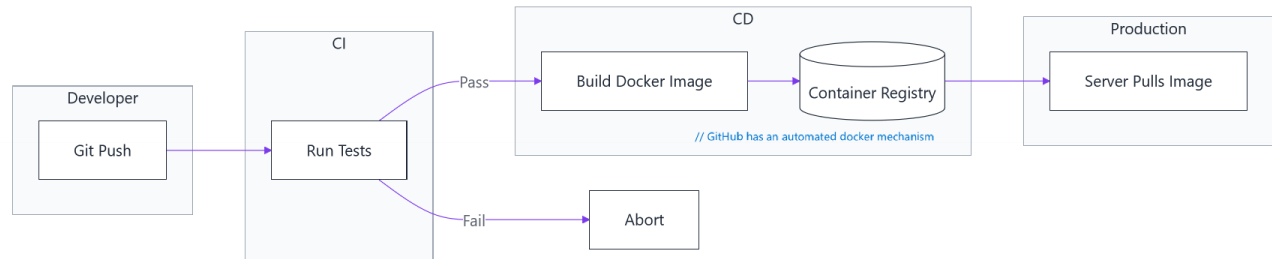
Automatically package verified code into standardized, deployable artifacts.

3. Continuous Deployment

Automatically push that package to end users (or a production environment), with safeguards to reduce manual error.

Anatomy of a Deployment Pipeline

A deployment pipeline consists of ordered, automated stages that move your code closer to users while enforcing checks at each step. If any check fails, the pipeline stops and prevents the defective release from progressing.



1. **Test:** Prove the logic works (`pytest`).
2. **Build:** Package the environment + code (`docker build`) into a self-contained image.
3. **Store:** Save the built image securely (Container Registry).
4. **Deploy:** Server downloads the image and starts running it.

Enforcing the Quality Gate

The Human Vulnerability

Having CI that runs `pytest` is not enough. If developers can merge changes regardless of test results, broken behavior flows into the shared `main` branch.

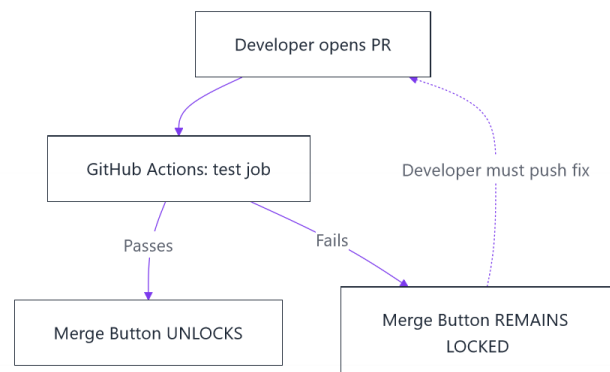
A test suite must function like an **unbypassable roadblock**.

Branch Protection Rules (GitHub)

You can lock down the `main` branch by requiring:

- No direct pushes (all changes go through Pull Requests)
- **Status checks to pass** before merging (CI tests succeed)
- The Merge button becomes disabled whenever checks fail

The Automated Bouncer



Source Code vs. Build Artifacts

The Recipe

Source Code (Git repository)

- `app.py`, `Dockerfile`, `requirements.txt`
- Instructions on how to build an application
- Fragile: requires compilation, dependency downloads, environment setup to run

The Baked Cake

Build Artifact (Docker Image)

- Self-contained, read-only snapshot
- OS + dependencies + your application code
- Runs without configuration or external downloads (if built correctly)

The Deployment Anti-Pattern

Never do this on a production server:

1. `git pull origin main`
2. `pip install -r requirements.txt`
3. `docker build .`

Why?

- Wastes server CPU/RAM and increases failure risk
- A dependency repository (PyPI) going down interrupts deployment
- **Environmental Drift:** succeeded on your laptop, failed on the server (different OS, missings, networking)

The "Build Once" Principle

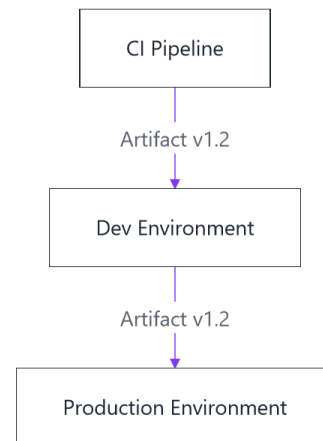
Eliminating Environmental Drift

In professional release engineering, promote **artifacts**, not source code.

1. The CI pipeline builds the Docker image **exactly once** (as part of the automated workflow).
2. That exact image is tested.
3. That exact image is deployed to every environment.

If you rebuild on dev, then rebuild again on production, you risk creating two different images.

Promoting Artifacts



What was tested is exactly what is deployed.

The Container Registry

Registry is to Docker as Repo is to Git

What is a Registry?

Secure, central storage for Docker images.
Think of it as "GitHub for compiled code."

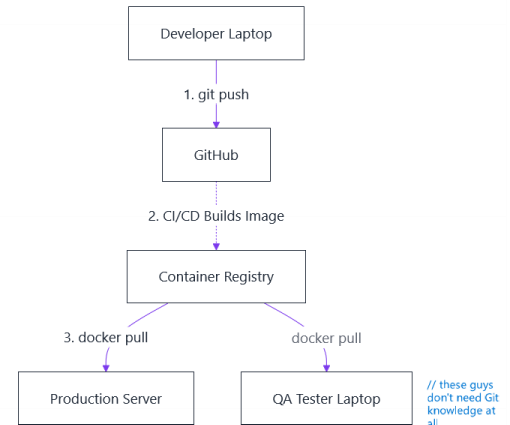
Standard Registry Options

- **Docker Hub** (Default public registry) this was where all the real softDevs were... but is now a weaker platform than the other 2
- **AWS Elastic Container Registry (ECR)** (Enterprise-grade, private)
- **GitHub Container Registry (GHCR)** (Integrated with repositories) // gitHub is just the goat

How it fits

Share image location instead of repositories:

```
docker pull ghcr.io/username/my-flask-app:latest
```



Your production server only talks to the Registry. It doesn't need to know GitHub exists.

Naming Artifacts: Image Tags

Anatomy of an Image Name

```
ghcr.io / my-username / my-app : latest
```

1. **Registry URL:** Where it lives
2. **Namespace:** Who owns it
3. **Repository:** The application
4. **Tag:** Which specific version

The Problem with `:latest`

`latest` is a moving target. If production crashes, you don't know which version caused it, and you cannot reliably roll back (the previous version is overwritten).

Tagging with Git SHAs

In CI/CD, tag Docker images with the **Git Commit SHA** (e.g., `a1b2c3d`).

Why this works:

1. **Traceability:** Look at a running container to identify exactly which source created it.
2. **Instant Rollbacks:** If `a1b2c3d` shows a defect, change the deployment to `f9e8d7c` and restart services.

Dependability requires instant reversion to a known good state.

Continuous Delivery: Implementing the Pipeline

often stored in a yml file

Add a new job to build and push the Docker image:

```
build-and-publish:
  needs: test # CRITICAL: Only run if tests pass!
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest

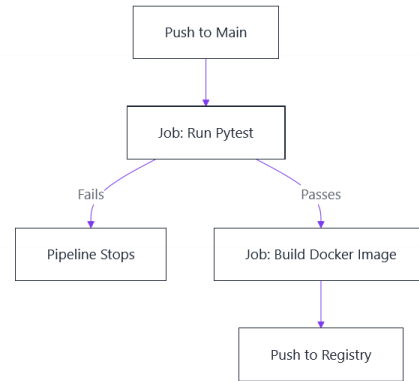
  permissions:
    contents: read
    packages: write

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Log in to Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
```

The Logic of `needs: test`

By making the publish job depend on test success, you guarantee that **every image stored in the registry has passed all tests**, thereby making artifact promotion safe and auditable.



Production Orchestration

`docker-compose.yml` (Local Development)

Optimized for fast iteration and debugging.

```
services:
  web:
    build: .
    volumes:
      - ./app
    environment:
      - FLASK_ENV=development
```

`docker-compose.prod.yml` (Production)

Optimized for stability, auditable deployments, & controlled exec.

```
services:
  web:
    image: ghcr.io/username/repo:a1b2c3d
    environment:
      - FLASK_ENV=production
```

Key Differences

1. **No `build: .`** — Use `image:` to pull the pre-tested artifact.
2. **No `volumes:`** — Production code must not change dynamically.
3. **Environment Variables** — Production behavior (disable debuggers, point to live services).

To deploy on a server:

```
docker compose -f docker-compose.prod.yml up -d
```

Cloud Deployment: From Local to Public

all 3 of these work really well for Flask apps, but remember you are on a FEATURE FREEZE!!

The Deployment Progression

- 1. **Local Machine:** Code runs on a laptop (development only)
- 2. **Docker Container:** Standardized environment, still local
- 3. **Cloud-Hosted Container:** Immutable artifact running on managed infrastructure (PaaS)

Why PaaS?

platform as a service

PaaS removes server overhead:

- No OS updates or server maintenance
- Automatic SSL/TLS certificates (HTTPS is the default)
- Auto-scaling (if needed) and simplified logging/monitoring

PaaS Comparison (Free Tier-Oriented)

PLATFORM	EASE OF DEPLOYMENT	DOCKER SUPPORT	ENVIRONMENT VARIABLE MANAGEMENT
Render	Very easy (GUI/CLI)	Full support	Secure dashboard secrets
Railway	Easy (drag-and-drop)	Full support	Project-level env vars
Fly.io	Moderate (CLI-first)	Full support	CLI secrets management

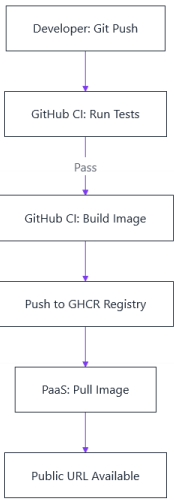
All three integrate with GitHub for automated deployments.

Cloud Deployment Workflow

Critical Production Considerations

- 1. **Environment Variables:** Configure secrets via the PaaS dashboard. **Never commit secrets to GitHub.**
- 2. **SSL/TLS:** PaaS providers auto-issue HTTPS certificates; production must not be served over HTTP.
- 3. **Local vs Cloud Differences:**
 - Connect to live databases (no emulators) // ps: for developing, it would have been best to use a persistent Firestore emulator, so u could do unlimited reads/writes
 - Network latency can affect real-time features
 - Free tier resource limits can cause timeouts

The full end-to-end pipeline for public deployment:



Requirements Traceability: Closing the Loop

The SDLC is a Loop

Software creation is not a line:

- 1. **Define** user needs and system requirements
- 2. **Translate** needs into implementable specifications
- 3. **Build** features, test code, containerize
- 4. **Verify** the deployed system solves the original problems

Requirements Traceability Matrix (RTM)

A lightweight tool to connect each user need to a feature and a test that confirms it is real.

Example RTM

USER REQUIREMENT	IMPLEMENTED FEATURE	ACCEPTANCE TEST
"View real-time data"	Dashboard + Chart.js polling	Load dashboard, verify chart updates
"Configure alert thresholds"	Threshold API + UI form	Set threshold, confirm alert triggers
"Store historical data"	Database CRUD endpoints	POST → GET → verify data persistence

Acceptance Testing vs. Unit Testing

Core Distinction

UNIT TESTING	ACCEPTANCE TESTING
Tests individual code components	Tests the full system from user perspective
Proves code works as written	Proves the system solves the user's problem
Isolated (mocks/stubs) mock: tests data... stub: tests functions against each other (good for integration testing)	Uses real services (live DB, cloud API)
Fast (seconds)	Slower (end-to-end setup required)

Why Both Matter

Unit tests catch internal defects early; acceptance tests confirm the delivered product matches user value and requirements.

Executing the Test Plan

Formal Acceptance Testing Process:

- 1. Launch the cloud-deployed application
- 2. Execute each RTM acceptance test step by step
- 3. Record pass/fail status and any deviations
- 4. If failed, document the root cause (API error, UI bug, permission issue, timeout)

Summary: From Local to Live

Key Concepts

- **Quality Gates:** Branch protection enforces that only tested code reaches `main`
- **Source vs. Artifact:** Version control stores the recipe; the Container Registry stores the “baked cake”
- **Build Once:** Promote the exact same immutable artifact through environments to eliminate drift
- **Cloud Deployment:** PaaS (Render, Railway, [Fly.io](#)) hosts containers publicly with automatic SSL/TLS
- **Secret Management:** Use PaaS dashboards for environment variables; never store secrets in GitHub
- **Requirements Traceability:** Map requirements to features and acceptance tests in an RTM
- **Acceptance Testing:** Prove the deployed system solves real problems, not just that code runs

You have transitioned from writing code to engineering dependable, distributed systems.

#app.py (lab 11)

File Edit Selection View Go Run Terminal Help

lab-11-CodeNube737

EXPLORER

LAB-11-CODENUBE737

- > __pycache__
- > blueprints
 - > __pycache__
- > api
 - > __pycache__
 - > __init__.py
- > routes.py
- > auth
 - > __init__.py
- > dashboard
- > profile
 - > __init__.py
- > decorators
- > test_api_profile.py
- > test_api_sensor.py
- > test_validators.py
- > utils
 - > __pycache__
 - > __init__.py
 - > auth.py
 - > logging_config.py
 - > profile.py
 - > validation.py
- > venv
- > .dockerignore
- > .env
- > .gitignore
- > app.py
- > config.py
- > docker-compose.yml
- > Dockerfile
- > firebase.py
- > mock_sensor_data.json
- > myResults.pdf
- > pytest.ini
- > Readme.md
- > requirements.txt
- > serviceAccountKey.json
- > summary.md

app.py

```
1 from flask import Flask
2
3
4 from blueprints.api import api_bp
5 from blueprints.auth import auth_bp
6 from blueprints.dashboard import dashboard_bp
7 from blueprints.profile import profile_bp
8 from config import Config
9 import firebase # noqa: F401
10
11 # Initialize structured logging
12 from utils import logging_config # noqa: F401
13
14 app = Flask(__name__)
15 app.config.from_object(Config)
16
17 app.register_blueprint(dashboard_bp)
18 app.register_blueprint(auth_bp)
19 app.register_blueprint(profile_bp)
20 app.register_blueprint(api_bp)
21
22 if __name__ == "__main__":
23     app.run(host="0.0.0.0", port=5000)
24
```

Dockerfile

```
1 # Use official Python 3.11 slim image
2 FROM python:3.11-slim
3
4 # Set working directory
5 WORKDIR /app
6
7 # Install dependencies first for better caching
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 # Copy the rest of the application code
12 COPY . .
13
14 # Expose port 5000 for Flask
15 EXPOSE 5000
16
17 # Default command to run the app
18 CMD ["python", "app.py"]
19
```

docker-compose.yml

```
1 version: "3.8"
2
3 services:
4   web:
5     build: .
6     ports:
7       - "5000:5000"
8     env_file:
9       - .env
10     volumes:
11       - ./app
12     environment:
13       - FLASK_DEBUG=1
14
```

pytest.ini

```
1 [pytest]
2 pythonpath = .
3 testpaths = tests
4
```

Folder Structure

```
code/
├─ src/      # Flask application source
├─ docs/     # Documentation, design notes, and diagrams
├─ tests/    # Automated and manual test cases
└─ README.md
```

Inside `src/`

The application follows a **blueprint-based modular architecture** — each feature area lives in its own package with clear boundaries.

```
src/
├─ app.py      # App factory – init, config, and blueprint registration only
├─ config.py   # Centralised constants (paths, secrets, TTLS)
├─ firebase.py # Firestore client initialisation
├─
├─ blueprints/
│  ├─ auth/    # Login, signup, logout & session handling
│  ├─ dashboard/ # Landing page and authenticated home view
│  ├─ device/  # Device page, live state & command API
│  ├─ media/   # Image upload, gallery listing & download
│  ├─ profile/ # User profile CRUD (Firestore-backed)
│  └─ notifications/ # Notification inbox, read/clear & demo presets
├─
├─ utils/
│  ├─ auth.py  # User persistence, password helpers, session lookup
│  ├─ firestore.py # Shared datetime / Firestore serialisation helpers
│  └─ notifications.py # Notification service, channels & publisher
├─
├─ decorators/ # (Planned) @login_required route guards
├─ templates/  # Jinja2 HTML templates
├─ static/     # CSS, JS, and static assets
└─ uploads/    # User-uploaded media (git-ignored)
```

#app.py (lab8, before separation of concerns)

```
from __future__ import annotations
from functools import wraps
from typing import Optional, Tuple, Union
from flask import Flask, render_template, request, redirect, url_for, session, jsonify, Response
from flask.typing import ResponseReturnValue
import firebase_admin
from firebase_admin import credentials, firestore, auth
from firebase_admin.firestore import DocumentReference
import os
import requests

app = Flask(__name__)
app.config["SECRET_KEY"] = os.getenv("FLASK_SECRET_KEY", "dev-secret-key")

WEB_API_KEY = os.environ.get("FIREBASE_WEB_API_KEY")

"""
# A dummy user for the login... i replaced every other instance of 'username' with 'email', so this is no longer needed.
# okay, so it should actually be the uid, not the email
dummy_user = {
    "username": "student",
    "password": "secret"
}
"""

# Initialize Firestore
if not firebase_admin._apps:
    service_account_path = os.getenv("FIREBASE_SERVICE_ACCOUNT", "serviceAccountKey.json")
    cred = credentials.Certificate(service_account_path)
    firebase_admin.initialize_app(cred)
db = firestore.client()

def get_current_user():
    """Return the currently logged-in email (or None).

    Uses session data set during `login`. This keeps all login checks
    consistent in one place.
    """
    if not session.get("logged_in"):
        return None
    return session.get("email")

def get_user_or_401():
    """Return the current API user (uid) or an Unauthorized response."""
    # Try to get JWT from Authorization header
    auth_header = request.headers.get("Authorization")
    if not auth_header or not auth_header.startswith("Bearer "):
        return jsonify({"error": "Unauthorized: Missing or invalid Authorization header"}), 401
    token = auth_header.split(" ", 1)[1]
    try:
        decoded_token = auth.verify_id_token(token)
        return decoded_token["uid"]
    except Exception as e:
        return jsonify({"error": f"Unauthorized: {str(e)}"}), 401

def get_profile_doc_ref(uid: str):
    """Get the Firestore document reference for a user's profile by UID."""
    return db.collection("profiles").document(uid)

def get_profile_data(uid: str):
    """Fetch a user's profile from Firestore, returning an empty dict if missing."""
    doc = get_profile_doc_ref(uid).get()
    return doc.to_dict() if doc.exists else {}

def validate_profile_data(first_name: str, last_name: str, student_id: str):
    """Validate that required profile fields are present and well-formed."""
    if not first_name or not last_name or not student_id:
        return "All fields are required."
    return None

def normalize_profile_data(first_name: str, last_name: str, student_id: str):
    """Normalize profile field values (strip whitespace, stringify student_id)."""
    return {
        "first_name": first_name.strip() if first_name else "",
        "last_name": last_name.strip() if last_name else "",
        "student_id": str(student_id).strip() if student_id else ""
    }

def require_json_content_type():
    """Ensure the request is JSON; returns an error response tuple if not."""
    if not request.is_json:
        return jsonify({"error": "Content-Type must be application/json"}), 415
    return None

def set_profile(uid: str, profile_data: dict[str, str], *, merge: bool):
    """Persist profile data to Firestore by UID.

    Args:
        uid: Profile owner (Firebase UID).
        profile_data: Data to write.
        merge: When True, merges into existing document (partial update).
    """
    get_profile_doc_ref(uid).set(profile_data, merge=merge)
    return None

def set_sensor_value(sensor_id: str, value: float, timestamp: str):
    db.collection("sensors").document(sensor_id).set({
        "value": value,
        "timestamp": timestamp
    })
    return None
```

```

def get_sensor_value(sensor_id: str):
    doc = db.collection("sensors").document(sensor_id).get()
    if doc.exists:
        return doc.to_dict()
    return {"value": None, "timestamp": None}

# --- Web Routes ---

@app.route("/signup", methods=["GET", "POST"])
def signup():
    """Signup page: create a new user and profile."""
    if request.method == "GET":
        return render_template("signup.html")

    email = request.form.get("email")
    password = request.form.get("password")
    confirm_password = request.form.get("confirm_password")

    if password != confirm_password:
        return render_template("signup.html", error="Passwords do not match")

    try:
        user = auth.create_user(email=email, password=password)
        db.collection("profiles").document(user.uid).set({
            "email": email,
            "role": "user",
        })
        return redirect(url_for("login"))
    except Exception as e:
        return render_template("signup.html", error=f"Signup failed: {str(e)}")

@app.route("/signup", methods=["POST"])
def api_signup():
    """API endpoint to create a user and profile."""
    data = request.get_json(silent=True) or {}
    email = data.get("email")
    password = data.get("password")
    confirm_password = data.get("confirm_password")

    if password != confirm_password:
        return jsonify({"error": "Passwords do not match"}), 400

    try:
        user = auth.create_user(email=email, password=password)
        db.collection("profiles").document(user.uid).set({
            "email": email,
            "role": "user",
        })
        return jsonify({"message": "Signup successful"}), 200
    except Exception as e:
        return jsonify({"error": f"Signup failed: {str(e)}"}), 400

@app.route("/api/login", methods=["POST"])
def api_login():
    """API endpoint to log in and return JWT."""
    data = request.get_json(silent=True) or {}
    email = data.get("email")
    password = data.get("password")
    if not email or not password:
        return jsonify({"error": "Email and password required"}), 400
    url = f"https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key={WEB_API_KEY}"
    payload = {"email": email, "password": password, "returnSecureToken": True}
    res = requests.post(url, json=payload)
    if res.status_code == 200:
        return jsonify({"token": res.json()["idToken"]}), 200
    return jsonify({"error": "Invalid credentials"}), 401

@app.route("/")
def home():
    """Home page. Redirects to login if no active session. Displays user profile info and sensor value if logged in."""
    current_user = get_current_user()
    if current_user:
        # Get UID for current_user (email)
        try:
            user_record = auth.get_user_by_email(current_user)
            uid = user_record.uid
        except Exception:
            uid = None
        profile_data = get_profile_data(uid) if uid else {}
        sensor_data = get_sensor_value("sensor-001")
        return render_template("dashboard.html", username=current_user, profile=profile_data, sensor=sensor_data)
    return redirect(url_for("login"))

@app.route("/login", methods=["GET", "POST"])
def login():
    """Login page using Firebase REST API for authentication."""
    if request.method == "GET":
        return render_template("login.html")

    email = request.form.get("email") # Email field
    password = request.form.get("password")

    if not email or not password:
        return render_template("login.html", error="Email and password required.")

    url = f"https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key={WEB_API_KEY}"
    payload = {"email": email, "password": password, "returnSecureToken": True}
    res = requests.post(url, json=payload)
    if res.status_code == 200:
        session["logged_in"] = True
        session["email"] = email
        return redirect(url_for("home"))
    return render_template("login.html", error="Invalid credentials. Try again.")

@app.route("/logout")
def logout():
    """Clear the session and return to login."""
    session.clear()
    return redirect(url_for("login"))

```

```

@app.route("/profile", methods=["GET", "POST"])
def profile():
    """HTML form to create/update the current user's profile using UID as key."""
    current_user = get_current_user()
    if not current_user:
        return redirect(url_for("login"))

    # Get UID for current_user (email)
    try:
        user_record = auth.get_user_by_email(current_user)
        uid = user_record.uid
    except Exception:
        uid = None

    if request.method == "GET":
        profile_data = get_profile_data(uid) if uid else {}
        return render_template("profile.html", profile=profile_data, error=None)

    first_name = request.form.get("first_name", "")
    last_name = request.form.get("last_name", "")
    student_id = request.form.get("student_id", "")

    error = validate_profile_data(first_name, last_name, student_id)
    if error:
        profile_data = {"first_name": first_name, "last_name": last_name, "student_id": student_id}
        return render_template("profile.html", profile=profile_data, error=error)

    normalized = normalize_profile_data(first_name, last_name, student_id)
    set_profile(uid, normalized, merge=False)
    return redirect(url_for("home"))

# --- API Routes ---

@app.get("/api/profile")
def api_get_profile():
    """Return the current user's profile."""
    user_or_response = get_user_or_401()
    if not isinstance(user_or_response, str):
        return user_or_response

    uid = user_or_response
    profile_data = get_profile_data(uid)
    return jsonify({"uid": uid, "profile": profile_data}), 200

@app.post("/api/profile")
def api_create_profile():
    """Create/replace the current user's profile from a JSON body."""
    user_or_response = get_user_or_401()
    if not isinstance(user_or_response, str):
        return user_or_response

    uid = user_or_response
    content_error = require_json_content_type()
    if content_error:
        return content_error

    data = request.get_json(silent=True) or {}
    first_name = data.get("first_name", "")
    last_name = data.get("last_name", "")
    student_id = data.get("student_id", "")

    error = validate_profile_data(first_name, last_name, student_id)
    if error:
        return jsonify({"error": error}), 400

    normalized = normalize_profile_data(first_name, last_name, student_id)
    set_profile(uid, normalized, merge=False)
    return jsonify({"message": "Profile saved successfully", "profile": normalized}), 200

@app.put("/api/profile")
def api_update_profile():
    """Update the current user's profile from a JSON body with robust validation."""
    user_or_response = get_user_or_401()
    if not isinstance(user_or_response, str):
        return user_or_response

    uid = user_or_response
    content_error = require_json_content_type()
    if content_error:
        return content_error

    data = request.get_json(silent=True) or {}
    if not data:
        return jsonify({"error": "Request body cannot be empty"}), 400

    allowed_fields = {"first_name", "last_name", "student_id"}
    errors = []
    update_data = {}

    # Whitelist check
    for key in data:
        if key not in allowed_fields:
            errors.append(f"Field '{key}' is not allowed.")

    # Bounds checking
    first_name = data.get("first_name")
    last_name = data.get("last_name")
    student_id = data.get("student_id")

    if first_name is not None:
        if len(first_name.strip()) > 50:
            errors.append("First name must be ≤ 50 characters.")
        update_data["first_name"] = first_name.strip()
    if last_name is not None:
        if len(last_name.strip()) > 50:
            errors.append("Last name must be ≤ 50 characters.")
        update_data["last_name"] = last_name.strip()
    if student_id is not None:
        sid = str(student_id).strip()
        if not (sid.isalnum() and len(sid) in [8,9]):
            errors.append("Student ID must be 8 or 9 alphanumeric characters.")
        update_data["student_id"] = sid

    if errors:
        return jsonify({"errors": errors}), 400
    if not update_data:

```

```

        return jsonify({"error": "No updatable fields provided"}), 400

    set_profile(uid, update_data, merge=True)
    updated_profile = get_profile_data(uid)
    return jsonify({"message": "Profile updated successfully", "profile": updated_profile}), 200

@app.delete("/api/profile")
def api_delete_profile():
    """Delete the current user's profile."""
    user_or_response = get_user_or_401()
    if not isinstance(user_or_response, str):
        return user_or_response

    uid = user_or_response
    get_profile_doc_ref(uid).delete()
    return jsonify({"message": "Profile deleted successfully"}), 200

def require_api_key(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        expected_key = os.environ.get("SENSOR_API_KEY")
        provided_key = request.headers.get("X-API-Key")
        if not provided_key or provided_key != expected_key:
            return jsonify({"error": "Unauthorized"}), 401
        return f(*args, **kwargs)
    return decorated_function

@app.route("/api/sensor_data", methods=["POST"])
@require_api_key
def sensor_data():
    data = request.get_json(silent=True) or {}
    sensor_id = data.get("sensor_id", "sensor-001")
    value = data.get("value")
    timestamp = data.get("timestamp")
    set_sensor_value(sensor_id, value, timestamp)
    return jsonify({"received": data}), 200

@app.route("/api/sensor_value", methods=["GET"])
def api_get_sensor_value():
    sensor_id = request.args.get("sensor_id", "sensor-001")
    sensor_data = get_sensor_value(sensor_id)
    return jsonify(sensor_data), 200

#-----
# Future work: Sensor Assignment & Access Control
# These dummy functions are for future implementation and do NOT affect current app behavior.
#
# Firestore changes required:
# - Add a 'sensors' field (list of sensor IDs) to each user profile document.
#   Example: { ..., "sensors": ["sensor-001", "sensor-002"] }
# - Optionally, create a 'sensor_permissions' collection to map sensors to allowed UIDs.

def assign_sensor_to_user(uid: str, sensor_id: str):
    """
    FUTURE: Assign a sensor to a user by adding the sensor_id to their profile's 'sensors' list in Firestore.
    Firestore change: Add or update the 'sensors' field in the user's profile document.
    """
    # Example Firestore update (not active):
    # db.collection("profiles").document(uid).update({"sensors": firestore.ArrayUnion([sensor_id])})
    pass

def get_user_sensors(uid: str):
    """
    FUTURE: Retrieve the list of sensor IDs assigned to a user from their profile document.
    Firestore change: Ensure 'sensors' field exists in profile documents.
    """
    # Example Firestore read (not active):
    # doc = db.collection("profiles").document(uid).get()
    # return doc.to_dict().get("sensors", []) if doc.exists else []
    pass

def user_can_access_sensor(uid: str, sensor_id: str) -> bool:
    """
    FUTURE: Check if a user has permission to access a given sensor.
    Firestore change: Use 'sensors' field in profile, or a 'sensor_permissions' collection.
    """
    # Example logic (not active):
    # sensors = get_user_sensors(uid)
    # return sensor_id in sensors
    pass

def remove_sensor_from_user(uid: str, sensor_id: str):
    """
    FUTURE: Remove a sensor assignment from a user in Firestore.
    Firestore change: Update the 'sensors' field in the user's profile document.
    """
    # Example Firestore update (not active):
    # db.collection("profiles").document(uid).update({"sensors": firestore.ArrayRemove([sensor_id])})
    pass

#-----
if __name__ == "__main__":
    app.run(debug=True, port=5000)

```